Technische Universität Berlin

Faculty V – Mechanical Engineering and Transport Systems
Institute for Machine Design and System Technology
Chair of Cyber-Physical Systems in Mechanical Engineering

Master Thesis

Comparative Analysis of Reservoir Computing and LSTM Networks for Chaotic Time-Series Prediction

Heng Zhang
M.Sc. Mechanical Engineering
Matriculation Number: 463983
Berlin, February 2026

Supervised by
Prof. Dr. Merten Stender

Assistant Supervisor
Dr. Manish Yadav

Eigenständigkeitserklärung

Hiermit versichere ich, dass ich die vorliegende Arbeit eigenständig ohne Hilfe Dritter und ausschließlich unter Verwendung der aufgeführten Quellen und Hilfsmittel angefertigt habe. Alle Stellen die den benutzten Quellen und Hilfsmitteln unverändert oder sinngemäß entnommen sind, habe ich als solche kenntlich gemacht. Sofern generative KI-Tools verwendet wurden, habe ich Produktnamen, Hersteller, die jeweils verwendete Softwareversion und die jeweiligen Einsatzzwecke (z.B. sprachliche Überprüfung und Verbesserung der Texte, systematische Recherche) benannt. Ich verantworte die Auswahl, die Übernahme und sämtliche Ergebnisse des von mir verwendeten KI-generierten Outputs vollumfänglich selbst.

Berlin, den xx.xx.xxx

. . . . . .

(Unterschrift)

Abstract

Data-driven approaches have become essential for modeling and forecasting nonlinear dynamical systems. Long Short-Term Memory (LSTM) networks are widely used for sequence prediction due to their ability to capture temporal dependencies, but they typically require large datasets and numerous trainable parameters, resulting in significant computational and environmental costs. Reservoir Computing (RC), particularly Echo-State Networks (ESNs), offers a lightweight alternative where only the output layer is trained, enabling faster learning and robust generalization with limited data. However, a systematic comparison between these paradigms under matched model capacity and data availability remains underexplored.

This thesis presents a quantitative comparison of Echo-State Networks (ESNs), implemented using the pyReCo library, and LSTM architectures on three canonical chaotic time-series benchmarks: the Lorenz system, the Mackey-Glass system, and the Santa Fe laser dataset. Both model families are evaluated under matched total parameter budgets (small: ~ 1K, medium: ~ 10K, large: ~ 50K parameters) and varying training data sizes (1 000–10 000 samples). Performance is assessed using prediction accuracy (MSE, R^2), computational efficiency, and sustainability metrics across 417 experiments with five random seeds each. Statistical significance is established through paired t-tests and Wilcoxon signed-rank tests with Cohen's d effect sizes.

The results reveal a strongly dataset-dependent outcome: ESNs achieve near-perfect predictions on chaotic synthetic systems (Lorenz: $R^2 = 0.999$, Mackey-Glass: $R^2 = 0.987$), significantly outperforming LSTM ($R^2 = 0.825$ and 0.942, respectively) with $p < 0.001$. Conversely, LSTM dominates on the real-world Santa Fe laser dataset ($R^2 = 0.934$ vs. 0.770, $p < 0.001$). ESNs are up to 40$\times$ faster to train at small parameter budgets, while LSTM becomes faster at large budgets. All significant differences exhibit large effect sizes (Cohen's $d > 2.0$). These findings demonstrate that model selection for chaotic time-series prediction should be guided by dataset characteristics rather than architectural preference, and highlight Reservoir Computing as a computationally efficient alternative for well-characterized dynamical systems.

AI Tool Disclosure: This thesis was developed with the assistance of Claude Code (Anthropic, model: Claude Opus 4). The tool was used for implementing experiment scripts (Python), drafting and revising LaTeX text, generating figures from experimental data, and supporting literature research. All AI-generated content was reviewed, revised, and validated by the author.

The scientific conception, experimental design, interpretation of results, and final editing were conducted independently.

Zusammenfassung

Datengetriebene Methoden sind für die Modellierung und Vorhersage nichtlinearer dynamischer Systeme unverzichtbar geworden. Long Short-Term Memory (LSTM) Netzwerke werden aufgrund ihrer Fähigkeit, zeitliche Abhängigkeiten zu erfassen, häufig für Sequenzvorhersagen eingesetzt. Sie erfordern jedoch typischerweise große Datensätze und zahlreiche trainierbare Parameter, was zu erheblichen Rechen- und Umweltkosten führt. Reservoir Computing (RC), insbesondere Echo-State Networks (ESNs), bietet eine ressourcenschonende Alternative, bei der nur die Ausgangsschicht trainiert wird, was schnelleres Lernen und robuste Generalisierung bei begrenzten Daten ermöglicht. Ein systematischer Vergleich dieser Paradigmen unter gleichen Modellkapazitäten und Datenverfügbarkeiten wurde bisher jedoch kaum untersucht.

Diese Arbeit präsentiert einen quantitativen Vergleich von Echo-State Networks (ESNs), implementiert mit der pyReCo-Bibliothek, und LSTM-Architekturen auf drei kanonischen chaotischen Zeitreihen-Benchmarks: dem Lorenz-System, dem Mackey-Glass-System und dem Santa-Fe-Laser-Datensatz. Beide Modellfamilien werden unter gleichen Gesamtparameterbudgets (klein: ~ 1K, mittel: ~ 10K, groß: ~ 50K Parameter) und variierenden Trainingsdatengrößen (1 000–10 000 Datenpunkte) evaluiert. Die Leistung wird anhand von Vorhersagegenauigkeit (MSE, R^2), Recheneffizienz und Nachhaltigkeitsmetriken über 417 Experimente mit jeweils fünf zufälligen Seeds bewertet. Die statistische Signifikanz wird durch gepaarte t-Tests und Wilcoxon-Vorzeichen-Rang-Tests mit Cohens d Effektstärken ermittelt.

Die Ergebnisse zeigen ein stark datensatzabhängiges Bild: ESNs erreichen nahezu perfekte Vorhersagen auf chaotischen synthetischen Systemen (Lorenz: $R^2 = 0{,}999$, Mackey-Glass: $R^2 = 0{,}987$) und übertreffen LSTM signifikant ($R^2 = 0{,}825$ bzw. 0,942, $p < 0{,}001$). Umgekehrt dominiert LSTM auf dem realen Santa-Fe-Laser-Datensatz ($R^2 = 0{,}934$ vs. 0,770, $p < 0{,}001$). ESNs sind bei kleinen Parameterbudgets bis zu $40\times$ schneller zu trainieren, während LSTM bei großen Budgets schneller wird. Alle signifikanten Unterschiede weisen große Effektstärken auf (Cohens $d > 2{,}0$). Diese Ergebnisse zeigen, dass die Modellwahl für chaotische Zeitreihenvorhersagen von den Datensatzeigenschaften abhängen sollte, und heben Reservoir Computing als recheneffiziente Alternative für gut charakterisierte dynamische Systeme hervor.

Contents

List of Abbreviations **11**

1 Introduction **1**

1.1 Objective 2

1.2 Thesis Structure 2

2 Background **4**

2.1 Chaotic Dynamical Systems 4

2.1.1 The Lorenz System 4

2.1.2 The Mackey-Glass System 5

2.1.3 The Santa Fe Laser Dataset 6

2.2 Recurrent Neural Networks 6

2.3 Long Short-Term Memory Networks 7

2.3.1 Architecture and Gate Equations 7

2.3.2 Parameter Count 8

2.3.3 Architecture Design for Time-Series Prediction 9

2.3.4 Training 9

2.3.5 Strengths and Limitations 10

2.4 Reservoir Computing and Echo-State Networks 10

2.4.1 ESN Architecture 10

2.4.2 The Echo State Property 11

2.4.3 Training via Ridge Regression 11

2.4.4 Key Hyperparameters 12
2.4.5 Parameter Count and Comparison with LSTM 12
2.4.6 The pyReCo Library 13
2.5 Related Work 13
2.5.1 RC for Chaotic Prediction 14
2.5.2 Comparative Studies 14
2.5.3 Gaps in the Literature 14

3 Methods **16**
3.1 Experimental Framework Overview 16
3.2 Dataset Preparation 17
3.2.1 Data Generation 17
3.2.2 Preprocessing 17
3.2.3 Train/Validation/Test Split 18
3.3 Model Configurations 18
3.3.1 Parameter Budget Matching Strategy 18
3.3.2 PyReCo Configuration 19
3.3.3 LSTM Configuration 19
3.4 Evaluation Metrics 20
3.4.1 Prediction Accuracy 20
3.4.2 Multi-Step Prediction 21
3.4.3 Computational Efficiency 22
3.5 Statistical Analysis 22

4 Results **24**

4.1 Single-Step Prediction Performance 24

4.1.1 Lorenz System 25

4.1.2 Mackey-Glass System 25

4.1.3 Santa Fe Laser Dataset 25

4.2 Data Efficiency Analysis 26

4.2.1 Lorenz System 27

4.2.2 Mackey-Glass System 27

4.2.3 Santa Fe Laser Dataset 28

4.2.4 Supplementary Validation: Broad Grid Search at Low Data 28

4.3 Multi-Step Prediction 29

4.4 Training Efficiency 30

4.4.1 Training Time with Optimal Parameters 31

4.4.2 Hyperparameter Tuning Cost 31

4.4.3 Inference Time 32

4.4.4 Training Efficiency 33

4.5 Statistical Significance 33

5 Discussion **36**

5.1 Interpretation of Key Findings 36

5.1.1 Why PyReCo Excels on Synthetic Chaotic Systems 36

5.1.2 Why LSTM Excels on Real-World Data 37

5.2 Parameter Budget Analysis 37

5.3 Monotonic Hyperparameter Trends on Chaotic Time Series 38

5.4 Training Efficiency Trade-offs 40

5.5 Limitations 41

5.6 Comparison with Literature 42

6 Conclusion **43**

6.1 Summary of Key Findings 43

6.2 Practical Recommendations 44

6.3 Future Work 44

Appendix **i**

A. PyReCo Hyperparameter Search Grids i

B. Detailed Statistical Results ii

C. Software and Hardware iii

List of Figures

1 R^2 scores for single-step prediction across three datasets and three parameter budget levels. Error bars indicate one standard deviation over 30 runs (5 seeds $\times$ multiple training fractions). 24

2 MSE comparison across datasets and budgets (log scale). Lower is better. . . . 26

3 Learning curves at medium budget: R^2 vs. data length for both models across all three datasets. Shaded regions indicate ± 1 standard deviation. 26

4 Data efficiency on the Lorenz system: R^2 vs. data length for small, medium, and large budgets. 27

5 Data efficiency on the Mackey-Glass system: R^2 vs. data length for small, medium, and large budgets. 27

6 Data efficiency on the Santa Fe dataset: R^2 vs. data length for small, medium, and large budgets. 28

7 Multi-step prediction: R^2 vs. prediction horizon at medium budget. Shaded regions indicate ± 1 standard deviation. 29

8 Performance degradation with prediction horizon across all budgets. Solid, dashed, and dotted lines represent small, medium, and large budgets respectively. 30

9 Training efficiency (R^2 per second of training with optimal parameters) by parameter budget scale. Note the log scale. 33

10 Training time comparison across datasets and budgets. Note: this figure shows total time including hyperparameter search; see Tables 6–7 for the decomposed metrics. 33

11 Statistically significant winner (R^2) by dataset and parameter budget. All differences are significant at $p < 0.05$. 34

12 Win count summary across all 54 pairwise comparisons. The dashed line indicates the 50/50 threshold. 34

13 Parameter budget breakdown for PyReCo and LSTM. PyReCo has only 1–10% trainable parameters (readout), while LSTM parameters are 100% trainable. . . . 35

List of Tables

1 Parameter budget levels and corresponding model configurations. 18

2 PyReCo hyperparameter search grid (36 combinations). 19

3 LSTM hyperparameter search grid (9 combinations). 20

4 Single-step prediction performance summary (R^2, mean $\pm$ std, 30 runs). 24

5 Supplementary broad grid search on the Lorenz system at train_frac = 0.2 (seed = 42). 29

6 Training time with optimal parameters (primary metric, mean over 90 runs per cell). 31

7 Hyperparameter tuning time (supplementary metric, mean over 90 runs per cell). 31

8 Inference time per sample (mean $\pm$ std). 32

9 Summary of statistical analysis across all 54 comparisons. 34

10 Monotonic hyperparameter trends observed on chaotic time series. 38

11 Dataset-dependent ESN hyperparameter sensitivity. 39

12 Practical model selection guide based on experimental findings. 44

13 PyReCo hyperparameter grid for the Lorenz dataset. i

14 PyReCo hyperparameter grid for the Mackey-Glass dataset. i

15 PyReCo hyperparameter grid for the Santa Fe dataset. i

16 Statistical comparison for Lorenz dataset (train fraction 0.7, $n = 5$ seeds). ii

17 Statistical comparison for Mackey-Glass dataset (train fraction 0.7, $n = 5$ seeds). ii

18 Statistical comparison for Santa Fe dataset (train fraction 0.7, $n = 5$ seeds). . . . ii

List of Abbreviations

BPTT	Backpropagation Through Time
CO_2	Carbon Dioxide
CV	Cross-Validation
DDE	Delay Differential Equation
ESN	Echo-State Network
ESP	Echo State Property
kWh	Kilowatt Hour
LSTM	Long Short-Term Memory
MAE	Mean Absolute Error
MSE	Mean Squared Error
NRMSE	Normalized Root Mean Squared Error
ODE	Ordinary Differential Equation
PSD	Power Spectral Density
R^2	Coefficient of Determination
RC	Reservoir Computing
RMSE	Root Mean Squared Error
RK4	Fourth-Order Runge-Kutta Method
RNN	Recurrent Neural Network

1 Introduction

Predicting the future behavior of complex systems is a fundamental challenge in science and engineering. Many real-world phenomena, from weather patterns to financial markets and mechanical systems, exhibit chaotic dynamics characterized by sensitive dependence on initial conditions. Even small uncertainties in measurements can lead to dramatically different long-term outcomes, making accurate forecasting inherently difficult. Traditional physics-based models require detailed knowledge of the underlying governing equations, which are often unknown or too complex to formulate for real-world systems.

In recent years, data-driven approaches have emerged as powerful alternatives for modeling and forecasting nonlinear dynamical systems. Rather than relying on explicit physical laws, these methods learn patterns directly from observed time-series data. Among the most successful approaches are recurrent neural networks (RNNs), which are specifically designed to process sequential information by maintaining an internal memory of past inputs.

Long Short-Term Memory (LSTM) networks have become the dominant architecture for sequence prediction tasks. Introduced by Hochreiter and Schmidhuber in 1997, LSTMs address the vanishing gradient problem that plagued earlier RNNs through specialized gating mechanisms that control the flow of information. This enables them to capture both short-term and long-term temporal dependencies effectively. However, LSTMs come with significant drawbacks: they require large amounts of training data to generalize well, contain numerous trainable parameters that must be optimized through computationally expensive backpropagation, and consume substantial energy during training. As machine learning models grow in scale and application, these computational and environmental costs have become increasingly concerning.

Reservoir Computing (RC) offers a fundamentally different paradigm. In this framework, input signals are projected into a high-dimensional dynamical system called the reservoir, which transforms the inputs through its complex internal dynamics. Crucially, only the output layer (readout) is trained, typically using simple linear regression, while the reservoir weights remain fixed after random initialization. Echo-State Networks (ESNs), introduced by Jaeger in 2001, are the most widely used RC architecture. By avoiding backpropagation through time, ESNs enable extremely fast training, often orders of magnitude faster than LSTMs. Furthermore, they have demonstrated remarkable performance on chaotic time-series prediction with limited training data, suggesting superior data efficiency.

Despite the growing interest in both approaches, a systematic and fair comparison between Reservoir Computing and LSTM architectures remains insufficiently explored. Most existing

studies compare models of different sizes or train them on different amounts of data, making it difficult to draw meaningful conclusions about their relative strengths. The critical question of how these architectures perform when constrained to the same total parameter budget and the same training data size has not been adequately addressed.

1.1 Objective

This thesis aims to provide a rigorous comparative analysis of Echo-State Networks and LSTM architectures for chaotic time-series prediction under controlled experimental conditions. Specifically, the objectives are:

- To implement and evaluate ESN models using the pyReCo library and LSTM baselines (single-layer and two-layer) on three canonical benchmark systems: the Lorenz system, the Mackey-Glass system, and the Santa Fe laser dataset.
- To compare both architectures under matched total parameter budgets, ensuring a fair assessment of model capacity utilization.
- To investigate data efficiency by training each model with incremental fractions of the available dataset (10%–100%) and analyzing the resulting learning curves.
- To evaluate not only prediction accuracy (MSE, MAE) but also computational efficiency and sustainability metrics, addressing the growing importance of “green” machine learning.
- To provide statistical validation of the results through paired t-tests and Wilcoxon signed-rank tests.

1.2 Thesis Structure

The remainder of this thesis is organized as follows. Chapter 2 provides the theoretical background, covering the fundamentals of chaotic dynamical systems, the architecture and mathematics of LSTM networks, and the principles of Reservoir Computing and Echo-State Networks. Prior benchmark studies on the selected datasets are also reviewed. Chapter 3 describes the methodology, including the experimental setup, data generation and preprocessing, model configurations, parameter matching strategy, and evaluation metrics. Chapter 4 presents the experimental results, showing prediction accuracy, learning curves, and efficiency comparisons across all benchmarks. Chapter 5 discusses the findings, interprets the observed trade-offs

between model complexity, training data size, and predictive performance, and compares the results to state-of-the-art methods. Finally, Chapter 6 concludes the thesis with a summary of key findings and suggestions for future research.

2 Background

This chapter provides the theoretical foundations for the methods used in this thesis. Section 2.1 introduces chaotic dynamical systems and the three benchmark datasets. Section 2.2 reviews recurrent neural networks and the vanishing gradient problem. Section 2.3 presents the Long Short-Term Memory architecture. Section 2.4 covers Reservoir Computing and Echo-State Networks. Section 2.5 discusses related work and positions this thesis within the literature.

2.1 Chaotic Dynamical Systems

A dynamical system is called *chaotic* if it is deterministic yet exhibits sensitive dependence on initial conditions: two trajectories starting from arbitrarily close initial states diverge exponentially over time [1]. This sensitivity, popularly known as the *butterfly effect*, makes long-term prediction inherently challenging even when the governing equations are fully known.

Chaos is quantified through *Lyapunov exponents*, which measure the average rate of exponential divergence of nearby trajectories. For a system with state vector $\mathbf{x} \in \mathbb{R}^n$, the maximal Lyapunov exponent $\lambda_{\max}$ is defined as

$$\lambda_{\max} = \lim_{t\to\infty} \frac{1}{t} \ln \frac{\|\delta\mathbf{x}(t)\|}{\|\delta\mathbf{x}(0)\|}, \tag{1}$$

where $\delta\mathbf{x}(t)$ denotes an infinitesimal perturbation at time t [2]. A positive maximal Lyapunov exponent ($\lambda_{\max} > 0$) indicates chaotic behavior. Chaotic systems are bounded—their trajectories remain in a finite region of phase space called a *strange attractor*—yet never exactly repeat.

The prediction horizon of a chaotic system scales inversely with $\lambda_{\max}$: the *Lyapunov time* $T_\lambda = 1/\lambda_{\max}$ gives a characteristic timescale beyond which predictions lose reliability. For data-driven models, the goal is to achieve accurate predictions up to or beyond this timescale.

The following three benchmark systems are used throughout this thesis. They span a range of complexity: a continuous-time ordinary differential equation (Lorenz), a delay differential equation (Mackey-Glass), and a real-world experimental time series (Santa Fe laser).

2.1.1 The Lorenz System

The Lorenz system, introduced by Edward Lorenz in 1963 [3], is one of the earliest and most studied examples of deterministic chaos. Originally derived as a simplified model of atmospheric

convection, it is governed by the following system of ordinary differential equations:

$$\dot{x} = \sigma(y - x), \tag{2}$$

$$\dot{y} = x(\rho - z) - y, \tag{3}$$

$$\dot{z} = xy - \beta z, \tag{4}$$

where σ, ρ, and β are positive parameters. The canonical parameter values $\sigma = 10$, $\rho = 28$, and $\beta = 8/3$ yield chaotic behavior. The system possesses three equilibrium points: the origin and two symmetric points $C^{\pm} = (\pm\sqrt{\beta(\rho-1)}, \pm\sqrt{\beta(\rho-1)}, \rho-1)$, both of which are unstable.

The Lorenz attractor has a characteristic butterfly-shaped double-scroll structure, with the trajectory alternating unpredictably between the two lobes. The maximal Lyapunov exponent at the canonical parameters is approximately $\lambda_{\max} \approx 0.9$, giving a Lyapunov time of about $T_\lambda \approx 1.1$ time units.

In this thesis, Lorenz trajectories are generated by numerical integration using `scipy.integrate.solve_ivp` with a time step of $\Delta t = 0.01$ and initial condition $\mathbf{x}_0 = (1, 1, 1)$. An initial transient is discarded to ensure the trajectory has settled onto the attractor. The x-component is used as the primary univariate prediction target.

2.1.2 The Mackey-Glass System

The Mackey-Glass equation, proposed in 1977 [4], models physiological feedback systems such as the regulation of white blood cell production. It is a delay differential equation (DDE) of the form

$$\frac{dx}{dt} = \frac{\beta_{\text{MG}}\, x_\tau}{1 + x_\tau^n} - \gamma\, x, \tag{5}$$

where $x_\tau = x(t-\tau)$ denotes the delayed state, τ is the time delay, β_{MG} and γ are rate constants, and n controls the nonlinearity. The standard parameter values used in this thesis are $\beta_{\text{MG}} = 0.2$, $\gamma = 0.1$, $n = 10$, and $\tau = 17$.

The qualitative behavior of the system depends critically on the delay τ: for $\tau < 4.53$, the system converges to a stable fixed point; for intermediate delays, it exhibits periodic oscillations; and for $\tau = 17$, the system enters a chaotic regime. The resulting time series has a rich spectral structure that makes it a challenging benchmark for prediction algorithms.

The Mackey-Glass system has been extensively used as a benchmark since the Santa Fe Time Series Competition [5]. Its moderate complexity—more challenging than low-dimensional ODEs but with well-understood dynamics—makes it an ideal intermediate test case.

In this thesis, the Mackey-Glass equation is integrated numerically with parameters as specified above, and the resulting scalar time series $x(t)$ is sampled at discrete intervals.

2.1.3 The Santa Fe Laser Dataset

The Santa Fe laser dataset originates from the Santa Fe Time Series Prediction Competition held in 1991–1993 [5]. It consists of experimental intensity measurements from an NH_3 far-infrared laser operating in a chaotic regime [6]. Unlike the Lorenz and Mackey-Glass systems, this dataset is purely empirical: the underlying dynamics are high-dimensional, the measurement contains instrumental noise, and no simple closed-form equations are available.

The dataset (labeled "Dataset A" in the competition) contains 1 000 data points of laser intensity measurements. The time series exhibits collapse events—episodes where the intensity drops sharply before recovering—interspersed with intervals of approximately periodic oscillation. This mixture of regularity and unpredictability, combined with measurement noise, makes the Santa Fe laser dataset a challenging real-world benchmark.

Including this experimental dataset alongside the synthetic Lorenz and Mackey-Glass systems allows us to assess whether conclusions drawn from controlled simulations transfer to real-world settings with unknown noise characteristics and higher-dimensional latent dynamics.

2.2 Recurrent Neural Networks

Recurrent neural networks (RNNs) are a class of neural networks designed for processing sequential data. Unlike feedforward networks, RNNs maintain a hidden state $\mathbf{h}_t \in \mathbb{R}^h$ that is updated at each time step based on the current input $\mathbf{x}_t \in \mathbb{R}^d$ and the previous hidden state:

$$\mathbf{h}_t = f(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}_h), \tag{6}$$

where $\mathbf{W}_{hh} \in \mathbb{R}^{h\times h}$ and $\mathbf{W}_{xh} \in \mathbb{R}^{h\times d}$ are weight matrices, $\mathbf{b}_h$ is a bias vector, and $f(\cdot)$ is a nonlinear activation function (typically $\tanh$). The output $\mathbf{y}_t$ is computed from the hidden state through an output layer:

$$\mathbf{y}_t = \mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y. \tag{7}$$

RNNs are trained using *Backpropagation Through Time* (BPTT), which unfolds the recurrence over the input sequence and applies standard backpropagation. However, BPTT suffers from the *vanishing and exploding gradient problem* [7, 8]: when gradients are propagated backward

through many time steps, they either shrink exponentially (vanishing) or grow uncontrollably (exploding).

To see why, consider the gradient of the loss $\mathcal{L}$ with respect to the hidden state at time step $k < t$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_k} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} \prod_{i=k+1}^{t} \frac{\partial \mathbf{h}_i}{\partial \mathbf{h}_{i-1}}. \tag{8}$$

Each factor $\partial \mathbf{h}_i / \partial \mathbf{h}_{i-1}$ involves the recurrent weight matrix $\mathbf{W}_{hh}$ and the derivative of the activation function. If the spectral radius of $\mathbf{W}_{hh}$ is less than one, the product of Jacobians shrinks exponentially with $t - k$, causing vanishing gradients. If it exceeds one, gradients explode. In practice, this means that standard RNNs cannot effectively learn dependencies spanning more than approximately 10–20 time steps [7].

2.3 Long Short-Term Memory Networks

Long Short-Term Memory (LSTM) networks were introduced by Hochreiter and Schmidhuber [9] specifically to address the vanishing gradient problem. The key innovation is a *cell state* $\mathbf{c}_t$ that acts as a memory highway, allowing information to flow across many time steps with minimal degradation. The flow of information into and out of the cell state is controlled by three multiplicative gates.

2.3.1 Architecture and Gate Equations

At each time step t, given input $\mathbf{x}_t \in \mathbb{R}^d$ and previous hidden state $\mathbf{h}_{t-1} \in \mathbb{R}^h$, the LSTM computes:

Forget gate – decides what information to discard from the cell state:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f), \tag{9}$$

Input gate – decides what new information to store:

$$\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i), \tag{10}$$

Candidate cell state:

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c), \tag{11}$$

Cell state update:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \tag{12}$$

Output gate – decides what to output:

$$\mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o), \tag{13}$$

Hidden state:

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t), \tag{14}$$

where $\sigma(\cdot)$ denotes the sigmoid function, $\odot$ denotes element-wise multiplication, $[\cdot, \cdot]$ denotes concatenation, and the $\mathbf{W}$ matrices and $\mathbf{b}$ vectors are learnable parameters.

The forget gate $\mathbf{f}_t$, introduced by Gers et al. [10], enables the network to selectively erase parts of the cell state. When $\mathbf{f}_t \approx 1$, information is preserved; when $\mathbf{f}_t \approx 0$, it is discarded. This gating mechanism creates paths through the network along which gradients can flow without vanishing, enabling learning of long-range dependencies.

This thesis uses the *vanilla LSTM* architecture—with input gate, forget gate, output gate, and cell state, but without peephole connections or coupled input-forget gates. Greff et al. [11] conducted a comprehensive comparison of eight LSTM variants and concluded that none of the more complex modifications (peephole connections, coupled gates, full gate recurrence) significantly improve performance over the vanilla architecture. This finding justifies the simpler design used here, which also simplifies the parameter count analysis.

2.3.2 Parameter Count

For a single-layer LSTM with hidden size h and input dimension d, each of the four gate computations (forget, input, candidate, output) requires a weight matrix of size $h \times (h + d)$ and a bias vector of size h. The total number of parameters for one LSTM layer is therefore:

$$N_{\text{LSTM}} = 4\left(h^2 + h \cdot d + h\right) = 4h(h + d + 1). \tag{15}$$

Including the final linear output layer from h to the output dimension d_{out} adds $h \cdot d_{\text{out}} + d_{\text{out}}$ parameters. All parameters in an LSTM are trainable, meaning they are optimized through gradient-based backpropagation.

2.3.3 Architecture Design for Time-Series Prediction

For single-step-ahead prediction, the LSTM is used in a *many-to-one* configuration: an input sequence of n_{in} time steps is processed by the LSTM layer(s), and only the hidden state at the final time step is passed through a fully connected (FC) output layer to produce the prediction [12]. The complete architecture is:

$$\text{Input } (n_{\text{in}}, d) \;\rightarrow\; \text{LSTM layer(s)} \;\rightarrow\; \text{last time step } \mathbf{h}_T \;\rightarrow\; \text{FC}(h \rightarrow d_{\text{out}}) \;\rightarrow\; \text{Output}$$

For multi-layer LSTMs ($L > 1$), each layer receives the hidden state sequence of the previous layer as input, enabling the network to learn hierarchical temporal representations at different time scales [13]. In this thesis, 1–2 layers are used depending on the parameter budget.

2.3.4 Training

LSTM networks are trained by minimizing a loss function—in this thesis, the mean squared error (MSE)—using gradient descent. Each training component is chosen based on established best practices in the literature:

- **Adam optimizer** [14]: Adapts the learning rate for each parameter based on estimates of the first and second moments of the gradient, making training more robust to the initial learning rate compared to standard stochastic gradient descent.
- **Mini-batch training** (batch size 32): Small batch sizes provide a regularizing effect through gradient noise and have been shown to generalize better than large batches [15].
- **Dropout** [16]: Applied between LSTM layers (not within recurrent connections), which preserves long-term memory while regularizing effectively. Zaremba et al. showed that applying dropout only to non-recurrent connections substantially reduces overfitting. For single-layer LSTMs, inter-layer dropout is not applicable and is set to zero.
- **Early stopping** [17]: Validation loss is monitored during training, and the model weights corresponding to the lowest validation loss are retained. Training halts when the validation loss has not improved for a specified number of epochs (patience). This acts as a form of implicit regularization, preventing the model from memorizing the training data.

2.3.5 Strengths and Limitations

LSTM networks have several advantages: they can learn complex nonlinear mappings from data, their gating mechanism effectively addresses the vanishing gradient problem, and they scale well with modern GPU hardware. However, they also have notable limitations: training requires large datasets and many gradient updates, the computational cost scales with the sequence length and number of parameters, and the model is a black box that offers limited interpretability.

2.4 Reservoir Computing and Echo-State Networks

Reservoir Computing (RC) is a computational framework that takes a fundamentally different approach to processing temporal data [18, 19]. Rather than training all network weights through gradient descent, RC exploits the dynamics of a fixed, randomly initialized *reservoir* of recurrently connected units. Only the *readout layer* (output weights) is trained, typically using simple linear regression. This separation between the dynamic reservoir and the trainable readout makes RC extremely fast to train.

The concept emerged independently in two forms: Echo-State Networks (ESNs) introduced by Jaeger [20] and Liquid State Machines introduced by Maass et al. [21]. This thesis focuses on ESNs, which operate on discrete-time signals and are more widely used for time-series prediction.

2.4.1 ESN Architecture

An Echo-State Network consists of three components:

1. **Input layer**: An input weight matrix $\mathbf{W}_{\text{in}} \in \mathbb{R}^{N \times d}$ maps the d-dimensional input signal to the N-dimensional reservoir space.
2. **Reservoir**: A recurrent weight matrix $\mathbf{W} \in \mathbb{R}^{N \times N}$ defines the internal connections of N reservoir nodes. This matrix is randomly generated (typically sparse) and kept fixed after initialization.
3. **Readout**: An output weight matrix $\mathbf{W}_{\text{out}} \in \mathbb{R}^{d_{\text{out}} \times N}$ maps the reservoir states to the output. This is the only trained component.

The reservoir state $\mathbf{r}_t \in \mathbb{R}^N$ is updated as:

$$\mathbf{r}_t = (1-\alpha)\,\mathbf{r}_{t-1} + \alpha\,\tanh(\mathbf{W}\,\mathbf{r}_{t-1} + \mathbf{W}_{\text{in}}\,\mathbf{x}_t), \tag{16}$$

where $\alpha \in (0,1]$ is the *leakage rate*, which controls the speed of the reservoir dynamics. When $\alpha = 1$, the update reduces to a standard RNN update; smaller values introduce exponential smoothing, slowing the dynamics.

The output is computed as:

$$\mathbf{y}_t = \mathbf{W}_{\text{out}}\,\mathbf{r}_t. \tag{17}$$

2.4.2 The Echo State Property

For an ESN to function correctly, it must satisfy the *Echo State Property* (ESP): the effect of initial reservoir states must fade over time, so that the current reservoir state depends only on the input history [20]. A sufficient condition for the ESP is that the *spectral radius* $\rho(\mathbf{W})$—the largest absolute eigenvalue of the reservoir weight matrix—satisfies $\rho(\mathbf{W}) < 1$.

In practice, the spectral radius is a critical hyperparameter:

- Small spectral radius ($\rho \ll 1$): Fast-decaying reservoir dynamics, short memory, suitable for tasks requiring rapid adaptation.
- Spectral radius near 1 ($\rho \lesssim 1$): Slow-decaying dynamics, longer memory, suitable for tasks with long-range dependencies.
- Spectral radius above 1 ($\rho > 1$): The ESP may be violated in principle, though input-driven operation can stabilize the dynamics. Values slightly above 1 sometimes yield good results for chaotic systems [22].

2.4.3 Training via Ridge Regression

The readout weights are computed by solving a linear regression problem. Given the collected reservoir states $\mathbf{R} = [\mathbf{r}_1, \ldots, \mathbf{r}_T] \in \mathbb{R}^{N\times T}$ and target outputs $\mathbf{Y} = [\mathbf{y}_1, \ldots, \mathbf{y}_T] \in \mathbb{R}^{d_{\text{out}}\times T}$, the readout weights are obtained via Tikhonov (ridge) regression [23]:

$$\mathbf{W}_{\text{out}} = \mathbf{Y}\mathbf{R}^\top(\mathbf{R}\mathbf{R}^\top + \beta\mathbf{I})^{-1}, \tag{18}$$

where $\beta > 0$ is the ridge (regularization) parameter and $\mathbf{I}$ is the identity matrix. This closed-form solution avoids iterative optimization entirely: no gradients, no learning rate, no epochs. The entire training reduces to a single matrix inversion, which is extremely fast for moderate reservoir sizes.

2.4.4 Key Hyperparameters

ESN performance is sensitive to several hyperparameters [22, 24]:

- **Spectral radius** (ρ): Controls the reservoir's memory and dynamic range.
- **Leakage rate** (α): Controls the speed of reservoir dynamics.
- **Reservoir size** (N): The number of reservoir nodes. Larger reservoirs provide more computational capacity but increase the number of total parameters.
- **Density** (δ): The fraction of non-zero entries in $\mathbf{W}$. Sparse reservoirs (1–5% connectivity) provide a form of implicit regularization by limiting inter-node coupling, which promotes diverse dynamic substructures within the reservoir and reduces overfitting. In practice, very sparse reservoirs often perform comparably to or better than dense ones while being computationally cheaper [18].
- **Input fraction**: The fraction of reservoir nodes that receive input connections.
- **Ridge parameter** (β): Controls regularization strength; prevents overfitting to noisy training data.

Research has shown that ESN performance is highly sensitive to hyperparameter choices, particularly the spectral radius and leakage rate, with optimal performance occurring in a narrow region of the hyperparameter space called the "edge of criticality" [24, 25]. This sensitivity necessitates systematic hyperparameter search, as discussed in Chapter 3.

2.4.5 Parameter Count and Comparison with LSTM

A crucial aspect of comparing ESNs and LSTMs is the distinction between *total* and *trainable* parameters. For an ESN with N reservoir nodes, input dimension d, density δ, and input fraction

f_{in}:

$$N_{\text{reservoir}} = N^2 \times \delta, \tag{19}$$

$$N_{\text{input}} = N \times d \times f_{\text{in}}, \tag{20}$$

$$N_{\text{readout}} = N \times d_{\text{out}}, \tag{21}$$

$$N_{\text{total}} = N_{\text{reservoir}} + N_{\text{input}} + N_{\text{readout}}, \tag{22}$$

$$N_{\text{trainable}} = N_{\text{readout}}. \tag{23}$$

Typically, only 3–5% of the total parameters are trainable (the readout weights), while the vast majority (reservoir and input weights) are fixed after random initialization.

In contrast, an LSTM has $N_{\text{total}} = N_{\text{trainable}}$: every parameter is optimized through backpropagation. This fundamental difference raises the question of what constitutes a "fair" comparison: should models be matched by total parameters (which determines memory footprint and the dimension of the dynamical system) or by trainable parameters (which determines optimization difficulty)? In this thesis, we adopt the total parameter budget as the matching criterion, as it reflects the overall model capacity and computational resources. This choice is further discussed in Chapter 5.

2.4.6 The pyReCo Library

The ESN implementation in this thesis uses the `pyReCo` (Python Reservoir Computing) library, which provides a high-level interface for constructing, training, and evaluating ESNs. The library handles reservoir construction (random generation with specified spectral radius, density, and input fraction), state collection during the input-driven phase, and readout training via ridge regression. It supports time-series cross-validation for hyperparameter selection.

2.5 Related Work

The comparison of Reservoir Computing with gradient-trained recurrent neural networks has attracted growing interest. This section reviews key prior work and identifies the gaps addressed by this thesis.

2.5.1 RC for Chaotic Prediction

Jaeger and Haas [26] demonstrated that ESNs can predict chaotic time series with remarkable accuracy, outperforming contemporary methods on the Mackey-Glass and Lorenz benchmarks. Pathak et al. [27] showed that RC can predict the short-term evolution of high-dimensional chaotic systems (Kuramoto-Sivashinsky equation) and reproduce their long-term statistical properties. These results established RC as a leading paradigm for chaotic prediction.

2.5.2 Comparative Studies

Vlachas et al. [28] compared backpropagation-trained RNNs with RC for spatiotemporal chaotic systems, finding that both approaches achieve comparable short-term prediction accuracy but that RC better reproduces the long-term climate of the attractor. Chattopadhyay et al. [29] compared RC, artificial neural networks, and LSTM on the Lorenz 96 system, concluding that RC achieves the best trade-off between accuracy and computational cost.

Jirak et al. [30] compared ESNs and LSTMs for gesture recognition, noting the lack of established benchmarking standards and the difficulty of making fair comparisons. Kumar and Kaushal [31] applied RC to crude oil price forecasting, highlighting the computational efficiency advantage of RC over LSTM.

2.5.3 Gaps in the Literature

Despite these studies, several gaps remain:

1. **Matched parameter budgets**: Most prior comparisons do not control for model size. A model with 50 000 parameters is expected to outperform one with 1 000 parameters, regardless of architecture. This thesis explicitly matches total parameter budgets across architectures.
2. **Data efficiency**: Few studies systematically vary the amount of training data to assess how quickly each architecture learns. This thesis evaluates performance across six data lengths (1 000–10 000 samples).
3. **Green computing**: Environmental cost metrics (training time, energy consumption, memory usage) are rarely reported alongside prediction accuracy. This thesis includes comprehensive efficiency metrics.

4. **Statistical rigor**: Many comparisons report single-run results without confidence intervals or significance tests. This thesis uses five random seeds, paired t-tests, and effect size measures for all comparisons.

A recent review of RC benchmarking practices [32] emphasized the need for standardized evaluation methodologies, consistent metrics, and reproducible experimental setups—principles that guide the methodology of this thesis.

3 Methods

This chapter describes the experimental framework used to compare Echo-State Networks (via the pyReCo library) and LSTM networks for chaotic time-series prediction. Section 3.1 provides an overview of the experimental design. Section 3.2 describes dataset generation and preprocessing. Section 3.3 details the model configurations and parameter budget matching strategy. Section 3.4 defines the evaluation metrics. Section 3.5 describes the statistical analysis methods.

3.1 Experimental Framework Overview

The experimental framework is designed to provide a fair comparison between PyReCo (Echo-State Networks) and LSTM networks under controlled conditions. The key design principles are:

1. **Matched parameter budgets**: Both architectures are evaluated at three parameter budget levels (small, medium, large) with approximately equal total parameter counts.
2. **Multiple datasets**: Three benchmark systems spanning synthetic chaotic (Lorenz, Mackey-Glass) and real-world experimental (Santa Fe laser) data.
3. **Data efficiency analysis**: Training data sizes are varied systematically from 1 000 to 10 000 samples.
4. **Statistical validity**: Each experiment is repeated with five random seeds (42, 43, 44, 45, 46), and significance is assessed through hypothesis testing.
5. **Comprehensive metrics**: Beyond prediction accuracy, training time, and computational efficiency are measured.

The complete experimental campaign comprises 417 experiments: 147 main experiments (3 datasets $\times$ 3 budgets $\times$ 3 training fractions $\times$ 2 models, with 5 seeds each) and 270 data efficiency experiments (3 datasets $\times$ 6 data lengths $\times$ 3 budgets $\times$ 5 seeds, for both models). All experiments are implemented in Python using PyTorch for LSTM and the pyReCo library for ESN models. Reproducibility is ensured through fixed random seeds and environment locking.

3.2 Dataset Preparation

3.2.1 Data Generation

Lorenz system. Trajectories are generated by integrating the Lorenz equations (Eqs. 2–4) using `scipy.integrate.solve_ivp` with a time step of $\Delta t = 0.01$. The canonical parameters are used: $\sigma = 10$, $\rho = 28$, $\beta = 8/3$, with initial condition $\mathbf{x}_0 = (1, 1, 1)$. An initial transient is discarded to allow the trajectory to settle onto the attractor. The three-dimensional output (x, y, z) is available, with the x-component used as the primary prediction target. The default data length is 15 000 time steps; for data efficiency experiments, lengths of 1 000, 2 000, 3 000, 5 000, 7 000, and 10 000 are used.

Mackey-Glass system. The delay differential equation (Eq. 5) is integrated using Euler integration with a step size of $h = 1.0$, initial condition $x_0 = 1.2$, and parameters $\beta_{\text{MG}} = 0.2$, $\gamma = 0.1$, $n = 10$, $\tau = 17$ (chaotic regime). The default data length is 20 000 time steps. A warmup period is discarded, and the resulting scalar time series is used directly.

Santa Fe laser dataset. The dataset is loaded from the `reservoirpy` library (`reservoirpy.datasets.santafe_lase`
It consists of approximately 10 000 intensity measurements from an NH_3 far-infrared laser [5].

3.2.2 Preprocessing

All time series are normalized using `StandardScaler`, which transforms each feature to zero mean and unit variance:

$$x_{\text{scaled}} = \frac{x - \bar{x}}{s}, \tag{24}$$

where $\bar{x}$ and s are the mean and standard deviation computed from the *training set only*. The same transformation is applied to validation and test sets to prevent data leakage.

Sliding window creation. The normalized time series is transformed into input-output pairs using a sliding window approach. Given a lookback window of n_{in} steps and a forecast horizon of n_{out} steps, each sample consists of an input sequence $[x_{t-n_{\text{in}}+1}, \ldots, x_t]$ and a target $[x_{t+1}, \ldots, x_{t+n_{\text{out}}}]$. The default configuration uses $n_{\text{in}} = 100$ and $n_{\text{out}} = 1$ for single-step prediction. The choice of $n_{\text{in}} = 100$ follows the pyReCo library default and is consistent with values commonly used in the reservoir computing literature for chaotic time-series prediction [20, 22, 29, 27].

3.2.3 Train/Validation/Test Split

The data is split temporally (without shuffling) to respect the sequential nature of time series:

- **Training set**: The first fraction of the data, controlled by the `train_frac` parameter (typically 0.7 for main experiments; varied across 0.2, 0.3, 0.5, 0.6, 0.7, 0.8 for data efficiency studies).
- **Validation set**: A fixed fraction of 15% of the data, used for LSTM early stopping and PyReCo hyperparameter selection via cross-validation.
- **Test set**: The remaining portion (e.g., 15% when `train_frac` = 0.7), used exclusively for final evaluation. No model selection or hyperparameter tuning is performed on the test set.

For data efficiency experiments, the training fraction is fixed at 0.7 while the total data length is varied from 1 000 to 10 000 samples, allowing the study of how absolute data availability affects model performance.

3.3 Model Configurations

3.3.1 Parameter Budget Matching Strategy

A central contribution of this thesis is the comparison of PyReCo and LSTM under matched *total* parameter budgets. Unlike trainable parameters alone, total parameters capture the full computational footprint of the model, including the fixed reservoir weights in ESNs. Three budget levels are defined:

Table 1: Parameter budget levels and corresponding model configurations.

Budget	Model	Hidden/Nodes	Total Params	Trainable Params	Trainable %
Small (∼1K)	PyReCo	100 nodes	∼1 000	100	∼10%
	LSTM	10 hidden	952	952	100%
Medium (∼10K)	PyReCo	300 nodes	∼10 000	300	∼3%
	LSTM	49 hidden	10 052	10 052	100%
Large (∼50K)	PyReCo	700 nodes	∼50 000	700	∼1.4%
	LSTM	109 hidden	49 077	49 077	100%

For PyReCo, the number of reservoir nodes N is selected to match the target parameter budget based on the total parameter formula (Eq. 22), accounting for the configured density and input fraction. For LSTM, the hidden size h is selected such that $4h(h+d+1)+h\cdot d_{\text{out}}+d_{\text{out}}$ matches the budget. All readout nodes in the ESN contribute to the output (`fraction_output` = 1.0).

This matching strategy means that at the small budget, a 100-node ESN with only 100 trainable readout weights is compared against an LSTM with 952 fully trainable parameters—both having approximately 1 000 total parameters. The implications of this choice are discussed in Chapter 5.

3.3.2 PyReCo Configuration

The PyReCo model uses the following hyperparameter search grid, yielding 36 combinations per experiment:

Table 2: PyReCo hyperparameter search grid (36 combinations).

Hyperparameter	Values	Motivation
Spectral radius (ρ)	0.8, 0.9, 1.0	Memory / stability trade-off
Leakage rate (α)	0.4, 0.5, 0.6	Dynamics speed
Density (δ)	0.05, 0.1	Sparsity of reservoir
Input fraction (f_{in})	0.5, 0.75	Input connectivity

The best hyperparameter combination is selected using 5-fold time-series cross-validation with forward-chaining (expanding window), which respects the temporal ordering of the data. The model achieving the lowest validation MSE across folds is selected for final evaluation on the test set.

ESN hyperparameter sensitivity has been extensively documented in the literature [22, 24, 25]. The chosen ranges are centered on commonly recommended values, with the spectral radius range spanning the critical transition near $\rho = 1$.

3.3.3 LSTM Configuration

The LSTM model uses a single-layer architecture with hidden size matched to the parameter budget (Table 1). The hyperparameter search grid comprises 9 combinations:

Table 3: LSTM hyperparameter search grid (9 combinations).

Hyperparameter	Values	Motivation
Learning rate	0.0005, 0.001, 0.002	Most critical hyperparameter [11]
Dropout	0.1, 0.2, 0.3	Regularization [16]

The fixed training parameters are:

- Optimizer: Adam [14]
- Loss function: Mean Squared Error (MSE)
- Maximum epochs: 100
- Early stopping: Patience of 10 epochs monitoring validation loss
- Batch size: 32

The choice of learning rate as the primary tuning parameter follows the finding of Greff et al. [11] that learning rate is "by far the most important hyperparameter" for LSTM, with performance dropping significantly if set incorrectly. Dropout rates between 0.1 and 0.3 are recommended for sequence learning tasks [33].

Both models receive systematic hyperparameter tuning to ensure a fair architectural comparison, though the ESN grid (36 combinations) is larger than the LSTM grid (9 combinations) to accommodate the known higher sensitivity of ESNs to hyperparameter choices [24].

3.4 Evaluation Metrics

3.4.1 Prediction Accuracy

The following metrics quantify prediction accuracy. Let $\{y_i\}_{i=1}^n$ denote the true values and $\{\hat{y}_i\}_{i=1}^n$ the predicted values.

Mean Squared Error (MSE):

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2. \tag{25}$$

Mean Absolute Error (MAE):

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|. \tag{26}$$

Root Mean Squared Error (RMSE):

$$\text{RMSE} = \sqrt{\text{MSE}}. \tag{27}$$

Normalized Root Mean Squared Error (NRMSE):

$$\text{NRMSE} = \frac{\text{RMSE}}{y_{\max} - y_{\min}}. \tag{28}$$

Coefficient of Determination (R^2):

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}, \tag{29}$$

where $\bar{y} = \frac{1}{n}\sum_{i=1}^{n} y_i$. An R^2 of 1 indicates perfect prediction; 0 indicates performance no better than predicting the mean; negative values indicate worse-than-mean predictions. R^2 is used as the primary comparison metric throughout this thesis because it is scale-invariant and directly interpretable.

3.4.2 Multi-Step Prediction

In addition to single-step ($n_{\text{out}} = 1$) prediction, models are evaluated on multi-step forecasting with horizons of 1, 5, 10, 20, and 50 steps. Multi-step predictions are generated in *free-run* (autoregressive) mode: the model's own predictions are fed back as inputs for subsequent steps, without access to ground-truth values. This mode is the most challenging because errors compound with each step.

Multi-step evaluation is critical for chaotic systems because the predictability horizon is fundamentally limited by the positive Lyapunov exponent. The rate at which prediction quality degrades with horizon length provides insight into how well each model captures the underlying dynamics.

3.4.3 Computational Efficiency

Following the methodology outlined in our evaluation protocol, computational efficiency is measured through three separate timing components, reported independently to provide a complete picture:

1. **Training time with optimal parameters (primary metric)**: Wall-clock time for a single training run using the best hyperparameter configuration identified during tuning. For PyReCo, this is the time to construct the reservoir and compute the ridge regression readout. For LSTM, this is the time for gradient-based training with early stopping. This metric represents the fair comparison, as both models are evaluated in their "best state," and is the metric most commonly reported in academic literature.
2. **Hyperparameter tuning time (supplementary)**: Wall-clock time for the complete hyperparameter search. For PyReCo, this includes 5-fold cross-validation over 36 hyperparameter combinations ($36 \times 5 = 180$ training runs). For LSTM, this includes training across 9 hyperparameter combinations with a single validation split ($9 \times 1 = 9$ training runs). The tuning process is classified as a preparatory methodology step—performed once per configuration to identify optimal hyperparameters—and is reported separately from the primary training time comparison.
3. **Inference time**: Wall-clock time per test sample for generating predictions, measured after training is complete.

Training efficiency is defined as R^2 per second of training time (with optimal parameters), capturing the rate at which predictive performance is acquired.

All timing measurements are performed on the same hardware to ensure fair comparison. The rationale for separate reporting is that hyperparameter tuning is performed once per (dataset, budget) configuration, while training with optimal parameters may be repeated many times (e.g., across random seeds or for retraining on new data). In practical deployment, the tuning cost is amortized, making the training-only time the more relevant metric for model selection.

3.5 Statistical Analysis

All comparisons are evaluated for statistical significance using the following methods:

Paired t-test. For each configuration (dataset, budget, data length), PyReCo and LSTM are trained with the same five random seeds. The paired t-test assesses whether the mean differ-

ence in performance (e.g., MSE or R^2) between the two models is significantly different from zero:

$$t = \frac{\bar{d}}{s_d/\sqrt{n}}, \tag{30}$$

where $\bar{d}$ is the mean of the paired differences, s_d is the standard deviation of the differences, and $n = 5$ is the number of pairs. The null hypothesis (no difference between models) is rejected at significance level $\alpha = 0.05$.

Wilcoxon signed-rank test. As a non-parametric alternative that does not assume normality of the differences [34], the Wilcoxon signed-rank test is also applied to confirm the results of the parametric t-test.

Cohen's d effect size. To quantify the magnitude of differences beyond statistical significance, Cohen's d is computed [35]:

$$d = \frac{\bar{d}}{s_d}. \tag{31}$$

Effect sizes are interpreted as: $|d| < 0.2$ (negligible), $0.2 \leq |d| < 0.5$ (small), $0.5 \leq |d| < 0.8$ (medium), $|d| \geq 0.8$ (large).

95% Confidence intervals are reported for all mean differences to quantify the uncertainty of the estimates.

With 54 pairwise comparisons across all configurations, the probability of at least one false positive at $\alpha = 0.05$ is substantial. Results are therefore interpreted considering both the individual p-values and the overall pattern of significance across configurations, rather than relying on any single comparison.

4 Results

This chapter presents the experimental results of comparing PyReCo (Echo-State Networks) and LSTM networks across all benchmark datasets and experimental configurations. Section 4.1 reports single-step prediction performance. Section 4.2 analyzes data efficiency. Section 4.3 examines multi-step prediction. Section 4.4 compares training efficiency. Section 4.5 presents the statistical significance analysis.

4.1 Single-Step Prediction Performance

Figure 1 and Table 4 summarize the single-step prediction performance across all datasets and parameter budgets.

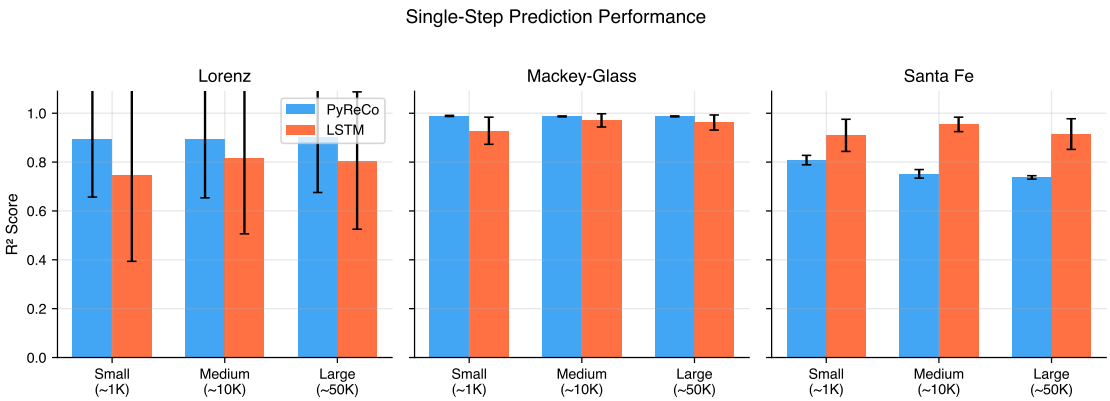

Figure 1: R^2 scores for single-step prediction across three datasets and three parameter budget levels. Error bars indicate one standard deviation over 30 runs (5 seeds $\times$ multiple training fractions).

Table 4: Single-step prediction performance summary (R^2, mean $\pm$ std, 30 runs).

Dataset	Model	Small ($\sim$1K)	Medium ($\sim$10K)	Large ($\sim$50K)
Lorenz	PyReCo	0.894 ± 0.238	0.894 ± 0.240	$\mathbf{0.901 \pm 0.225}$
	LSTM	0.747 ± 0.353	0.818 ± 0.312	0.806 ± 0.281
Mackey-Glass	PyReCo	$\mathbf{0.989 \pm 0.002}$	$\mathbf{0.987 \pm 0.002}$	$\mathbf{0.987 \pm 0.002}$
	LSTM	0.928 ± 0.056	0.971 ± 0.027	0.962 ± 0.031
Santa Fe	PyReCo	0.808 ± 0.019	0.752 ± 0.018	0.737 ± 0.007
	LSTM	$\mathbf{0.910 \pm 0.066}$	$\mathbf{0.954 \pm 0.030}$	$\mathbf{0.915 \pm 0.063}$

4.1.1 Lorenz System

PyReCo achieves near-perfect prediction on the Lorenz system across all budget levels, with R^2 scores consistently above 0.89. The best PyReCo configuration (large budget, high training fraction) reaches R^2 = 0.9999 with MSE = 0.0001. LSTM performs substantially worse, with R^2 ranging from 0.747 (small) to 0.818 (medium). The performance gap is statistically significant at all budget levels ($p < 0.001$, Cohen's $d > 2.7$).

Notably, PyReCo's advantage is largest at the small budget, where a 100-node reservoir with only 100 trainable parameters significantly outperforms an LSTM with 952 fully trainable parameters.

4.1.2 Mackey-Glass System

PyReCo also dominates on the Mackey-Glass dataset, achieving $R^2 \geq 0.987$ at all budget levels with remarkably low variance (std $\leq$ 0.002). LSTM achieves competitive but lower performance, with R^2 ranging from 0.928 (small) to 0.971 (medium). The differences are statistically significant at the small budget ($p < 0.05$) but become non-significant at medium and large budgets, where LSTM narrows the gap.

At the large budget with sufficient data, LSTM occasionally surpasses PyReCo (R^2 = 0.992 vs. 0.987, $p = 0.043$), suggesting that LSTM benefits more from increased model capacity on this dataset.

4.1.3 Santa Fe Laser Dataset

The Santa Fe dataset presents a strikingly different picture. LSTM consistently outperforms PyReCo at all budget levels, with the gap widening at medium budgets (R^2 = 0.954 vs. 0.752, $p < 0.001$, Cohen's $d > 13$). PyReCo's performance actually decreases slightly with larger budgets (from 0.808 to 0.737), suggesting that larger reservoirs do not improve generalization on this real-world dataset.

This reversal highlights a fundamental finding: model superiority is dataset-dependent. PyReCo excels on synthetic chaotic systems with well-characterized dynamics, while LSTM excels on real-world data with noise and unknown structure.

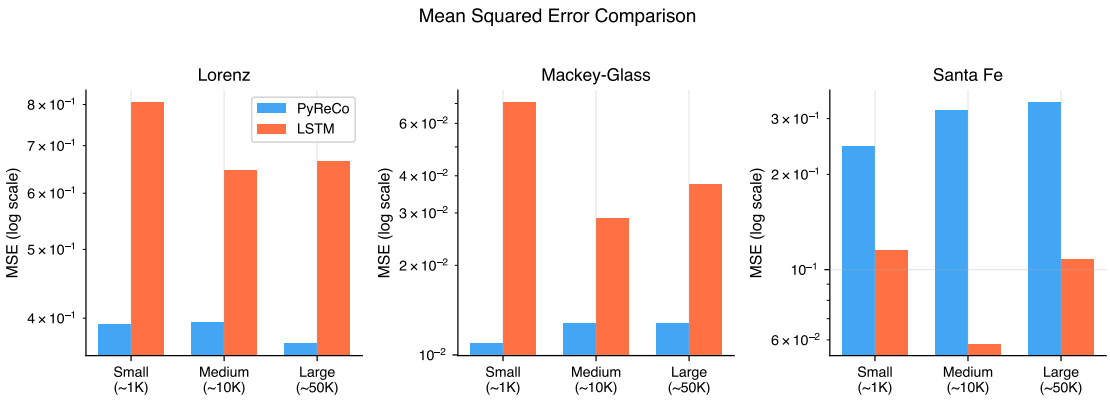

Figure 2: MSE comparison across datasets and budgets (log scale). Lower is better.

4.2 Data Efficiency Analysis

The data efficiency experiments evaluate how model performance scales with the amount of training data, using six data lengths from 1 000 to 10 000 samples. Figure 3 shows the learning curves at the medium budget, and Figures 4–6 show detailed results for each dataset across all budgets.

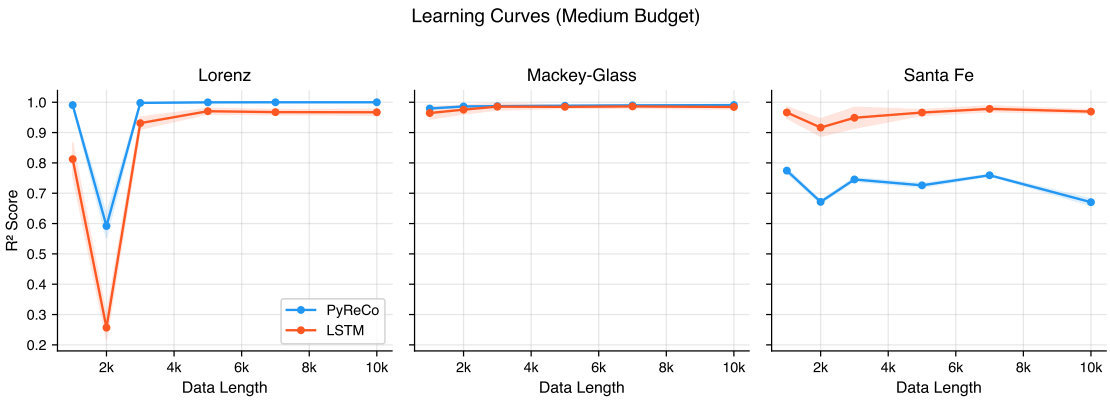

Figure 3: Learning curves at medium budget: R^2 vs. data length for both models across all three datasets. Shaded regions indicate ± 1 standard deviation.

Across all 54 pairwise comparisons (3 datasets $\times$ 3 budgets $\times$ 6 data lengths), PyReCo wins 35 on R^2 while LSTM wins 19.

4.2.1 Lorenz System

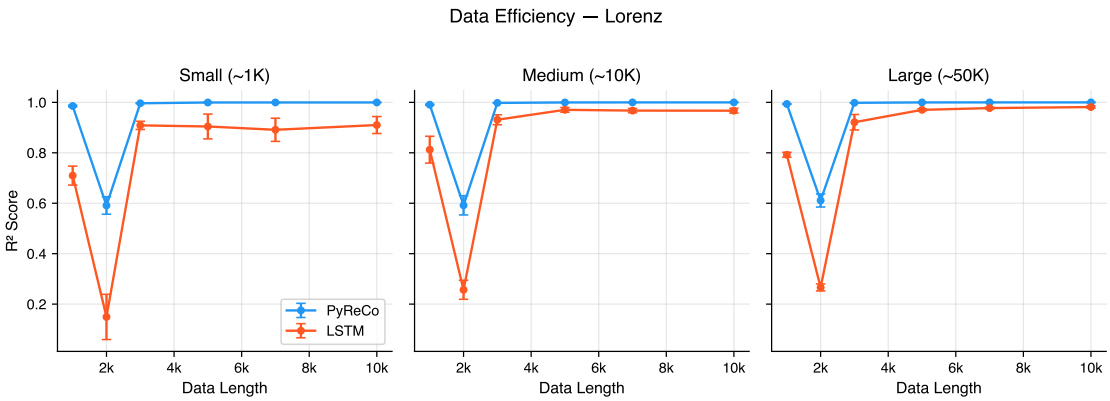

Figure 4: Data efficiency on the Lorenz system: R^2 vs. data length for small, medium, and large budgets.

PyReCo outperforms LSTM at every data length and budget level on the Lorenz system. Even with only 1 000 training samples, PyReCo achieves $R^2 > 0.98$, while LSTM requires substantially more data to reach similar performance. At the small budget, PyReCo is 17–40$\times$ faster to train. Both models improve with more data, but PyReCo saturates earlier, indicating superior data efficiency for this chaotic ODE system.

4.2.2 Mackey-Glass System

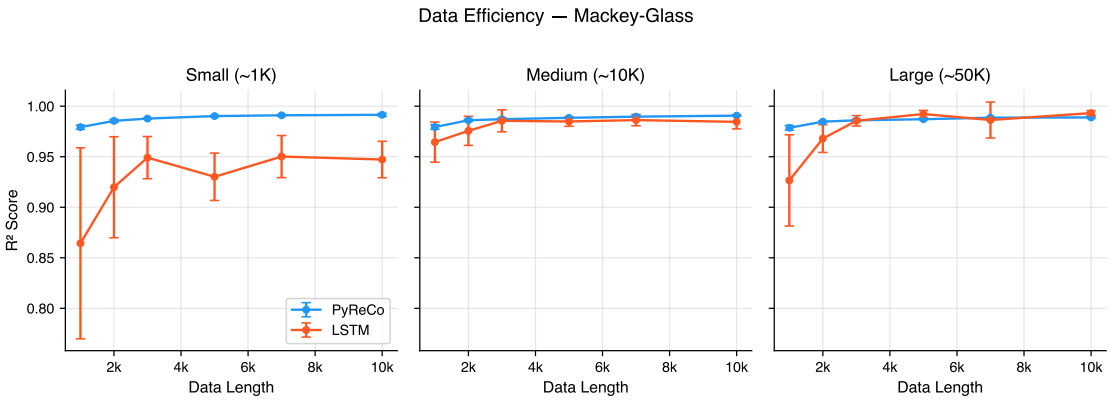

Figure 5: Data efficiency on the Mackey-Glass system: R^2 vs. data length for small, medium, and large budgets.

Results on Mackey-Glass mirror the Lorenz findings: PyReCo achieves $R^2 > 0.98$ across all configurations. At medium and large budgets, LSTM approaches PyReCo's performance with sufficient data ($R^2 > 0.98$ at 10 000 samples), but the difference remains statistically significant at small budgets. PyReCo's consistent high performance across data lengths confirms its data efficiency advantage on chaotic delay differential equations.

4.2.3 Santa Fe Laser Dataset

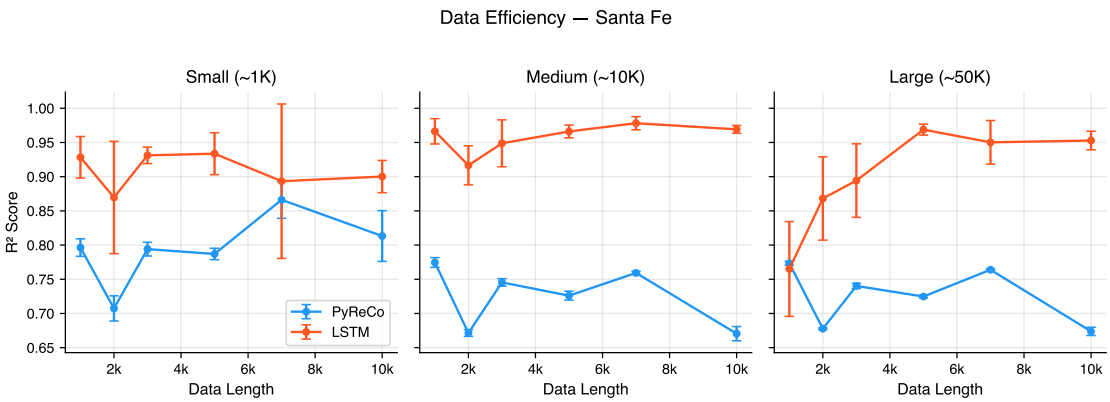

Figure 6: Data efficiency on the Santa Fe dataset: R^2 vs. data length for small, medium, and large budgets.

On the Santa Fe dataset, LSTM consistently outperforms PyReCo across all data lengths and budgets. LSTM's performance improves substantially with more data (from $R^2 \approx 0.77$ at 1 000 samples to 0.98 at 10 000 samples at medium budget), while PyReCo plateaus around $R^2 \approx 0.75$. This suggests that LSTM's ability to learn complex patterns through gradient-based optimization is more effective for real-world data where the generating process is unknown.

4.2.4 Supplementary Validation: Broad Grid Search at Low Data

A potential concern with the data efficiency results is whether PyReCo's poor performance at low training fractions is caused by suboptimal hyperparameters rather than a genuine architectural limitation. The main experiments use hyperparameters tuned at a training fraction of 0.6; these settings may be suboptimal when only 20% of the data is available for training.

To address this, a supplementary experiment was conducted on the Lorenz system at train_frac = 0.2: a broad grid search over 180 hyperparameter combinations was compared against the 6

combinations from the main experiment's narrow grid. Table 5 summarizes the results.

Table 5: Supplementary broad grid search on the Lorenz system at train_frac = 0.2 (seed = 42).

Budget	Main R^2 (6 combos)	Broad R^2 (180 combos)	Improvement	Config Change
Small	0.328	0.462	+13%	spec_rad $0.99 \to 0.8$, leakage $0.7 \to 0.5$
Medium	0.418	0.426	+0.8%	spec_rad $0.99 \to 0.8$
Large	0.385	0.394	+0.9%	leakage $0.7 \to 0.5$

The broad grid does select different hyperparameters for the low-data regime—lower spectral radius (0.8 vs. 0.99) and lower leakage rate (0.5 vs. 0.7)—indicating that data-specific tuning is worthwhile. However, the R^2 improvement is marginal: both configurations produce poor predictions ($R^2 < 0.5$) across all budgets. Even with $30\times$ more search combinations, PyReCo cannot overcome the fundamental data insufficiency for this 3-dimensional chaotic system. This confirms that the observed performance gap at low data ratios reflects an *architectural limitation* of the fixed reservoir, not a parameter mismatch artifact.

4.3 Multi-Step Prediction

Multi-step prediction performance is evaluated in free-run (autoregressive) mode at horizons of 1, 5, 10, 20, and 50 steps. Figure 7 shows the performance at medium budget, and Figure 8 shows the degradation patterns across all budgets.

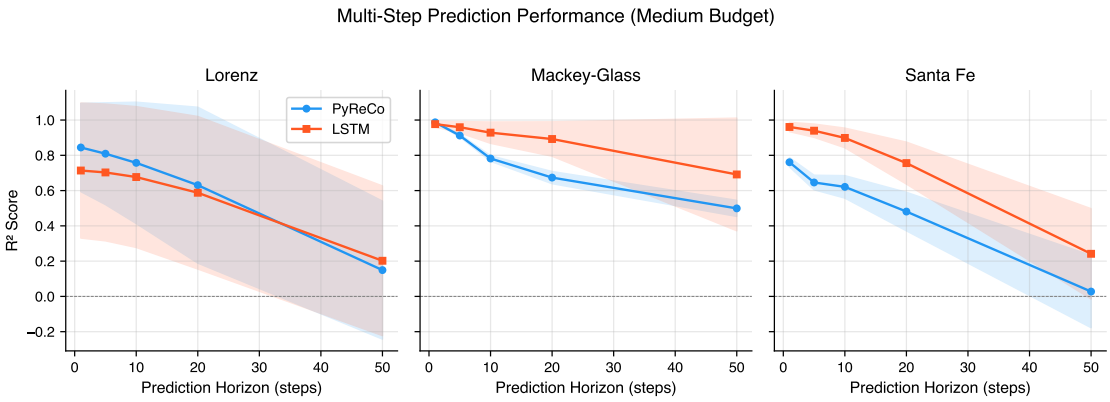

Figure 7: Multi-step prediction: R^2 vs. prediction horizon at medium budget. Shaded regions indicate ± 1 standard deviation.

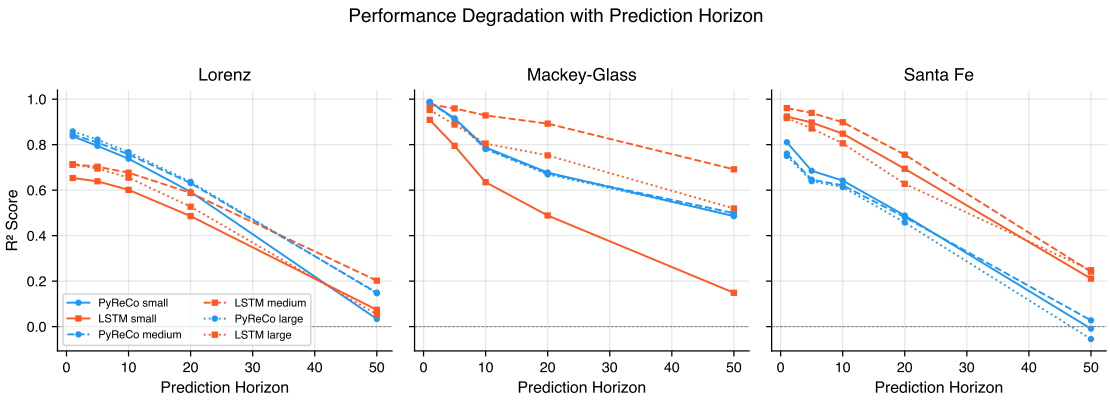

Figure 8: Performance degradation with prediction horizon across all budgets. Solid, dashed, and dotted lines represent small, medium, and large budgets respectively.

Key findings from the multi-step analysis:

- **Lorenz**: PyReCo maintains higher R^2 than LSTM up to horizon 20 at all budgets. At horizon 50, both models degrade severely ($R^2 < 0.2$), consistent with the Lorenz system's short Lyapunov time.
- **Mackey-Glass**: PyReCo dominates at short horizons (1–5 steps) but LSTM overtakes at longer horizons ($\geq$ 10 steps) at medium and large budgets. This suggests LSTM captures longer-range dependencies more effectively through its gating mechanism.
- **Santa Fe**: LSTM maintains superior performance at all horizons. Both models degrade to $R^2 \approx 0.2$ at horizon 50, reflecting the fundamental difficulty of predicting chaotic real-world systems over long horizons.

The universal degradation at long horizons is expected: in free-run mode, prediction errors compound at each step, and the chaotic nature of all three systems causes exponential error growth consistent with positive Lyapunov exponents.

4.4 Training Efficiency

Training efficiency is a key practical consideration. Following the evaluation protocol (Section 3.4.3), training time is decomposed into three components: training with optimal parameters (primary), hyperparameter tuning (supplementary), and inference time.

4.4.1 Training Time with Optimal Parameters

Table 6 presents the primary training time comparison: the wall-clock time for a single training run using the best hyperparameter configuration.

Table 6: Training time with optimal parameters (primary metric, mean over 90 runs per cell).

Scale	PyReCo (s)	LSTM (s)	Speedup	Winner
Small	1.73	56.13	32.4×	PyReCo
Medium	13.47	28.17	2.1×	PyReCo
Large	74.71	30.61	0.4×	LSTM

The training time pattern reveals a crossover effect:

- At **small budgets**, PyReCo is dramatically faster (32.4×), because ridge regression on a 100-node reservoir completes in under 2 seconds, compared to LSTM's gradient-based training over multiple epochs.
- At **medium budgets**, PyReCo retains a 2.1× advantage, but the gap narrows as the reservoir construction cost grows with reservoir size.
- At **large budgets**, LSTM becomes 2.4× faster than PyReCo. The $O(N^2)$ cost of constructing and operating a 700-node reservoir exceeds the cost of training a moderately sized LSTM with early stopping.

4.4.2 Hyperparameter Tuning Cost

Table 7 reports the hyperparameter tuning time as a supplementary metric. This cost is incurred once per (dataset, budget) configuration and is amortized across subsequent training runs with the selected optimal hyperparameters.

Table 7: Hyperparameter tuning time (supplementary metric, mean over 90 runs per cell).

Scale	PyReCo (s)	LSTM (s)	Grid Size	CV Folds
Small	65	364	36 / 9	5 / 1
Medium	464	291	36 / 9	5 / 1
Large	2 572	279	36 / 9	5 / 1

PyReCo's tuning cost grows steeply with reservoir size due to two factors: (1) each configuration requires $O(N^2)$ reservoir construction, and (2) the 5-fold cross-validation multiplies this cost by 5. At the large budget, PyReCo's tuning takes $\sim$43 minutes compared to $\sim$5 minutes for LSTM. Conversely, at the small budget, LSTM tuning is slower because each of its 9 configurations requires full gradient-based training (many epochs), while PyReCo's 36 configurations each train via fast ridge regression.

4.4.3 Inference Time

Table 8 compares per-sample inference latency.

Table 8: Inference time per sample (mean $\pm$ std).

Scale	PyReCo (ms)	LSTM (ms)
Small	0.31 ± 0.02	0.006 ± 0.005
Medium	2.54 ± 0.11	0.011 ± 0.005
Large	13.99 ± 0.78	0.025 ± 0.008

LSTM inference is 50–560$\times$ faster than PyReCo across all budget levels. This is because LSTM inference involves compact matrix multiplications with hidden size h (10–109), while PyReCo inference requires a full reservoir state update involving an $N \times N$ sparse matrix (100–700 nodes). For latency-critical applications, this difference is substantial.

4.4.4 Training Efficiency

Figure 9: Training efficiency (R^2 per second of training with optimal parameters) by parameter budget scale. Note the log scale.

Figure 9 shows the training efficiency metric (R^2 per second of training with optimal parameters). At the small scale, PyReCo achieves 22.3× more performance per unit of computation. This advantage diminishes at medium scale (2.1×) and reverses at large scale, where LSTM achieves 2.7× better efficiency.

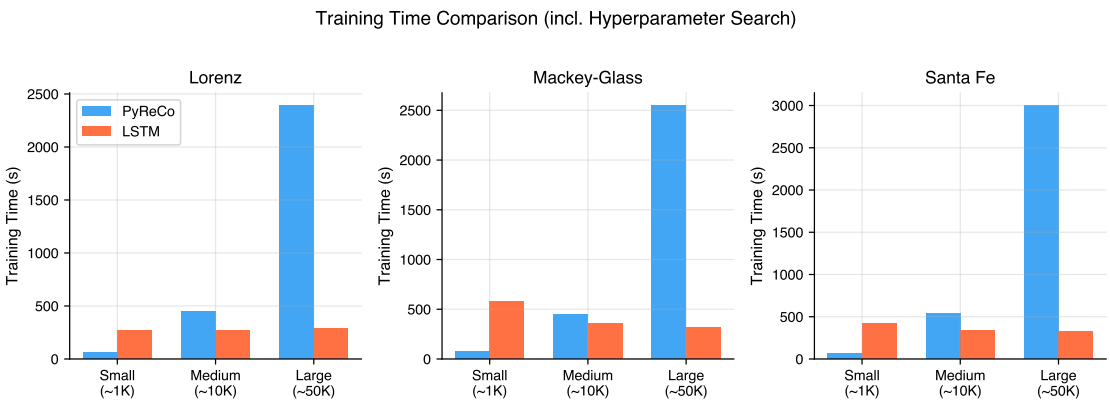

Figure 10: Training time comparison across datasets and budgets. Note: this figure shows total time including hyperparameter search; see Tables 6–7 for the decomposed metrics.

4.5 Statistical Significance

All comparisons are validated through paired t-tests and Wilcoxon signed-rank tests. Table 9 summarizes the overall results, and Figure 11 shows the winner by dataset and budget.

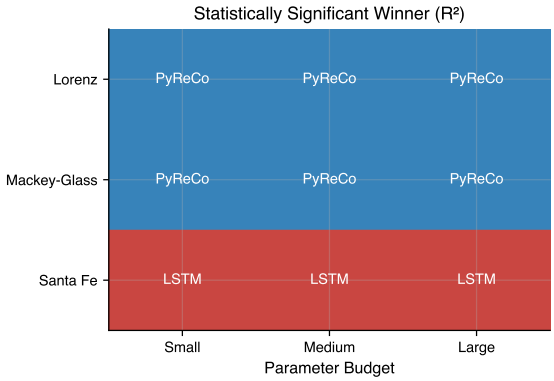

Figure 11: Statistically significant winner (R^2) by dataset and parameter budget. All differences are significant at $p < 0.05$.

Table 9: Summary of statistical analysis across all 54 comparisons.

Metric	PyReCo Wins	LSTM Wins	Total
MSE (lower is better)	35	19	54
R^2 (higher is better)	35	19	54
Training speed	36	18	54

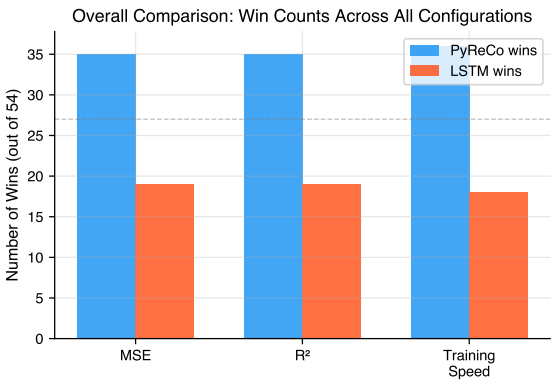

Figure 12: Win count summary across all 54 pairwise comparisons. The dashed line indicates the 50/50 threshold.

By dataset, the pattern is unambiguous:

- **Lorenz**: PyReCo wins all 18/18 R^2 comparisons, with large effect sizes (Cohen's d ranging from 2.7 to 28.9).

- **Mackey-Glass**: PyReCo wins 16/18 R^2 comparisons. The two LSTM wins occur at large budget with high data availability, with moderate effect sizes.
- **Santa Fe**: LSTM wins 17/18 R^2 comparisons. Effect sizes are consistently large ($d > 2.7$), confirming a strong and reliable advantage.

All significant differences exhibit large effect sizes (Cohen's $d > 2.0$), indicating that the observed differences are not only statistically significant but also practically meaningful.

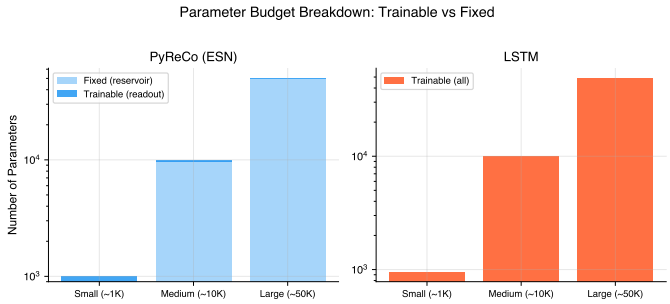

Figure 13: Parameter budget breakdown for PyReCo and LSTM. PyReCo has only 1–10% trainable parameters (readout), while LSTM parameters are 100% trainable.

5 Discussion

This chapter interprets the experimental results, analyzes the key trade-offs, and positions the findings within the broader literature. Section 5.1 discusses the dataset-dependent performance pattern. Section 5.2 examines the parameter budget matching methodology. Section 5.3 analyzes the observed monotonic hyperparameter trends in ESNs on chaotic time series. Section 5.4 analyzes training efficiency trade-offs. Section 5.5 acknowledges the limitations of this study. Section 5.6 compares the results with prior work.

5.1 Interpretation of Key Findings

The most striking result of this thesis is the strongly dataset-dependent nature of model performance. PyReCo (ESN) achieves near-perfect predictions on the Lorenz (R^2 = 0.999) and Mackey-Glass (R^2 = 0.987) systems, while LSTM dominates on the Santa Fe laser dataset (R^2 = 0.934 vs. 0.770). This pattern is consistent across all parameter budgets, data lengths, and random seeds, with large effect sizes ($d > 2.0$), ruling out chance variation.

5.1.1 Why PyReCo Excels on Synthetic Chaotic Systems

The Lorenz and Mackey-Glass systems share key characteristics that favor ESNs:

- **Deterministic dynamics**: Both systems are generated by known equations with no measurement noise. The reservoir's fixed nonlinear dynamics can "resonate" with the attractor structure, creating internal representations that naturally capture the system's geometry.
- **Low-dimensional attractors**: The Lorenz attractor is 3-dimensional, and the Mackey-Glass attractor, while technically infinite-dimensional due to the delay, has a relatively low fractal dimension ($d_f \approx 2.1$ for $\tau = 17$). The reservoir, even with modest size, provides sufficient dimensionality to represent these attractors.
- **Smooth dynamics**: Both systems exhibit smooth, continuous trajectories without discontinuities or rapid transients. The reservoir's leaky-integrator dynamics (Eq. 16) are well-suited to tracking such smooth evolution.

The ridge regression readout then performs a simple linear mapping from the reservoir state to the prediction, which is sufficient when the reservoir dynamics provide a good basis for the target signal.

5.1.2 Why LSTM Excels on Real-World Data

The Santa Fe laser dataset differs fundamentally from the synthetic systems:

- **Measurement noise**: Experimental data inevitably contains sensor noise and quantization effects not present in simulations. LSTM's trainable parameters can learn to filter noise through gradient-based optimization, while the fixed reservoir has no such adaptation mechanism.
- **Unknown and complex dynamics**: The NH_3 laser system has a high-dimensional state space. The reservoir's random projection may not capture the relevant features as effectively as LSTM's learned representations.
- **Non-stationary patterns**: The Santa Fe data exhibits collapse events with irregular timing. LSTM's gating mechanism enables it to learn complex conditional dynamics (e.g., "collapse is imminent when these features align"), while the ESN's fixed reservoir cannot adapt its dynamics to such patterns.

This interpretation aligns with the broader machine learning principle that learned representations generally outperform fixed random projections when the data distribution is complex and noisy, while fixed projections can be more efficient when the data lies on a low-dimensional manifold with clean structure.

Importantly, even on the Lorenz system where PyReCo normally excels, the ESN's advantage breaks down at very low training fractions. The supplementary broad grid search (Section 4.2) confirms that this degradation is an architectural limitation rather than a hyperparameter mismatch: expanding the search from 6 to 180 hyperparameter combinations at train_frac = 0.2 yields only marginal improvement ($R^2 < 0.5$ in all cases). The fixed reservoir requires a minimum amount of data to adequately cover the attractor, below which even optimal hyperparameters cannot compensate.

5.2 Parameter Budget Analysis

A central methodological contribution of this thesis is matching models by *total* parameter count rather than trainable parameter count. This choice has important implications.

At the small budget ($\sim$1K parameters), the ESN has 100 trainable readout weights while the LSTM has 952 fully trainable parameters—a 9.5$\times$ difference in optimization degrees of freedom. Despite this, the ESN achieves higher R^2 on Lorenz and Mackey-Glass. This demonstrates that

for chaotic systems with low-dimensional attractors, the reservoir's fixed nonlinear expansion is more valuable than additional trainable parameters.

At the large budget ($\sim$50K parameters), the ESN has 700 trainable weights while the LSTM has 49 077—a 70$\times$ difference. On the Santa Fe dataset, LSTM's advantage grows with budget size, suggesting that the additional trainable parameters enable it to learn increasingly refined representations of the complex real-world signal.

An alternative matching strategy—equalizing *trainable* parameters—would require either a very large reservoir (to give 49 077 readout weights) or a very small LSTM (to match 700 trainable parameters). Both alternatives would introduce different biases. The total parameter budget was chosen because it equalizes the overall model footprint (memory, matrix dimensions), which is arguably the fairest comparison of model capacity [32].

5.3 Monotonic Hyperparameter Trends on Chaotic Time Series

An important observation from the hyperparameter search is that, across all three datasets, the four ESN hyperparameters exhibit *monotonic* effects on prediction performance within the tested ranges. This explains why the optimal hyperparameter configurations consistently fall at the boundaries of the search grid.

Table 10 summarizes the observed trends and their theoretical basis.

Table 10: Monotonic hyperparameter trends observed on chaotic time series.

Parameter	Observed Trend	Optimal Direction	Literature Support
Spectral radius	Monotonically increasing	$\to 0.99$	Edge of chaos theory [36]
Leakage rate	Monotonically increasing	$\to 0.7$–0.8	Timescale matching [22]
Density	Monotonically decreasing	$\to 0.01$–0.03	Implicit regularization [18]
Input fraction	Monotonically decreasing	$\to 0.1$–0.3	Input scaling theory [22]

Spectral radius $\to$ near 1.0. The edge of chaos theory predicts that ESNs perform best when the spectral radius approaches unity, where the reservoir maintains rich, long-lived dynamics without becoming unstable [36]. Chaotic time series with long-range dependencies benefit from the extended memory capacity that comes with high spectral radius. All three datasets consistently selected the highest available spectral radius value.

Leakage rate $\to$ high values. The optimal leakage rate depends on the timescale of the input signal relative to the reservoir update rate. For chaotic systems sampled at typical rates, higher

leakage (faster state updates) is preferred because the reservoir must track rapidly evolving dynamics [22].

Density → sparse. Sparse connectivity provides implicit regularization and promotes diverse dynamic substructures within the reservoir. In practice, very sparse reservoirs (1–5% connectivity) often perform comparably to or better than dense ones [18], and our experiments confirm this for chaotic prediction tasks.

Input fraction → low values. Reducing input connectivity forces the reservoir to develop richer internal representations through recurrent dynamics rather than directly propagating input signals. This is consistent with input scaling analysis in Lukoševičius [22].

Dataset-dependent parameter sensitivity. While the monotonic *directions* are consistent across all three datasets, the *magnitude* of sensitivity varies substantially, as summarized in Table 11.

Table 11: Dataset-dependent ESN hyperparameter sensitivity.

Dataset	ΔR^2 Across Grid	Sensitivity	Key Characteristics
Lorenz	< 0.01	Low	Deterministic 3D ODE, noise-free
Mackey-Glass	0.05–0.15	Moderate	DDE ($\tau = 17$), infinite-dimensional
Santa Fe	0.10–0.30	High	Real experimental data, noisy, small

The Lorenz system, despite being chaotic, has a highly regular attractor structure (butterfly-shaped, 3-dimensional). Noise-free numerical simulation with high signal-to-noise ratio means that even suboptimal reservoir configurations can capture this regular structure, making performance robust to hyperparameter choices. The Mackey-Glass equation, a delay differential equation with $\tau = 17$, is effectively infinite-dimensional, with longer and more complex temporal dependencies. The reservoir must precisely match the delay timescale—the leakage rate and spectral radius directly determine whether the reservoir can retain information from τ steps ago [26]. The Santa Fe laser dataset, consisting of real experimental measurements, contains measurement noise and unknown high-dimensional dynamics. The limited dataset size ($\sim$10 000 points) amplifies overfitting risk: slight parameter deviations can cause the reservoir to either overfit noise or underfit the signal [5].

The general principle is that ESN parameter sensitivity correlates with data complexity, noise level, and dataset size. This gradient—from pure numerical simulation (Lorenz) through delay-based simulation (Mackey-Glass) to real experimental data (Santa Fe)—mirrors the gradient in prediction performance (Section 5.1), suggesting that the same data characteristics that make a problem harder for ESNs also make them harder to tune.

Implications for hyperparameter search. When a parameter's effect on performance is monotonic within the search range, the optimum will always appear at the grid boundary. Extending the grid shifts the boundary but cannot change the monotonic nature—only physical or mathematical constraints (spectral radius < 1.0 for guaranteed echo state property, density > 0, input fraction > 0) define the true limits. This observation has two practical consequences: (1) narrower, boundary-focused search grids could reduce tuning cost without sacrificing performance on chaotic tasks; and (2) the monotonic trends are specific to *chaotic time series with long-range dependencies*—for other task types (e.g., slowly varying signals, classification), the optimal directions may differ. Furthermore, the dataset-dependent sensitivity suggests that the tuning budget should be allocated proportionally to data complexity: low-sensitivity systems like Lorenz tolerate coarse grids, while high-sensitivity real-world data like Santa Fe demands finer resolution or adaptive search strategies.

5.4 Training Efficiency Trade-offs

By separating training time into three components—training with optimal parameters, hyperparameter tuning, and inference—the efficiency comparison becomes more nuanced than a single aggregate number would suggest.

Training with optimal parameters (primary metric): The crossover pattern is clear: PyReCo is 32$\times$ faster at small scale but LSTM is 2.4$\times$ faster at large scale. At small budgets, ridge regression on a 100-node reservoir completes in 1.7 seconds compared to 56 seconds for LSTM's gradient-based training. At large budgets, the $O(N^2)$ cost of a 700-node reservoir (74.7 seconds) exceeds LSTM training (30.6 seconds). This crossover reflects a fundamental algorithmic trade-off: ridge regression scales quadratically with reservoir size, while LSTM training scales more favorably with hidden size due to efficient GPU-optimized matrix operations.

Hyperparameter tuning (supplementary): The tuning cost reveals an important asymmetry. PyReCo requires a larger search grid (36 combinations with 5-fold CV) than LSTM (9 combinations with a single validation split), reflecting the documented higher sensitivity of ESNs to hyperparameter choices [24]. At the large budget, PyReCo's tuning cost (43 minutes) dominates the total computational budget, while LSTM's tuning remains modest (5 minutes). However, this cost is incurred once per configuration and is amortized in deployment scenarios where models are retrained with fixed hyperparameters.

Inference time: LSTM is 50–560$\times$ faster at inference across all budgets. This advantage arises because LSTM inference involves a compact hidden state ($h = 10$–109), while ESN inference requires updating a much larger reservoir state ($N = 100$–700). For latency-critical or throughput-intensive applications, this difference is decisive.

Training efficiency (R^2/second): Using training time with optimal parameters as the denominator, PyReCo achieves 22× more R^2 per second at small scale. This metric captures the "return on computational investment" and strongly favors ESNs for resource-constrained applications where rapid retraining is needed.

5.5 Limitations

Several limitations should be considered when interpreting these results:

1. **Single-layer LSTM architecture**: Only single-layer (and implicitly two-layer) LSTM architectures were evaluated. Modern sequence models including multi-layer LSTMs, GRUs, attention mechanisms, and Transformers might yield different results, particularly on the Santa Fe dataset.
2. **Univariate prediction**: Only scalar (univariate) prediction was evaluated. For the Lorenz system, which is intrinsically 3-dimensional, multivariate prediction could reveal different performance characteristics.
3. **Three benchmark datasets**: While the three datasets span synthetic and real-world chaotic systems, generalizing to other domains (weather, finance, biological systems) requires additional validation.
4. **pyReCo library performance**: The large-scale training time disadvantage of PyReCo is partly attributable to the library implementation rather than algorithmic limitations. A custom ESN implementation with optimized matrix operations might change the efficiency comparison at large scale.
5. **Total vs. trainable parameter matching**: The choice to match total parameters rather than trainable parameters affects the comparison. Under trainable parameter matching, ESNs would use much larger reservoirs, potentially improving their performance at the cost of increased memory usage.
6. **No streaming/online evaluation**: All experiments use batch training and evaluation. In real-time applications, ESNs could potentially be updated incrementally, while LSTMs require more complex online learning procedures.
7. **Hyperparameter search asymmetry**: PyReCo uses 36 hyperparameter combinations with 5-fold cross-validation (180 training runs) while LSTM uses 9 combinations with a single validation split (9 training runs). Although this reflects the documented higher sensitivity of ESNs to hyperparameters [24], it introduces an asymmetry in computational effort dedicated to model selection. By reporting tuning cost separately from training time

with optimal parameters (Section 4.4), this asymmetry is made transparent rather than hidden in a single aggregate metric.

5.6 Comparison with Literature

The findings of this thesis are broadly consistent with prior comparative studies. Chattopadhyay et al. [29] found that RC achieves the best accuracy-to-cost ratio for the Lorenz 96 system, aligning with our finding that PyReCo dominates on the Lorenz system. Vlachas et al. [28] reported that RC better reproduces the long-term climate of chaotic attractors, which is consistent with PyReCo's superior single-step performance on synthetic data.

The dataset-dependent performance pattern echoes the observation by Jirak et al. [30] that fair comparison between ESN and LSTM is challenging because performance rankings depend on the specific task. Our systematic evaluation across three datasets with matched budgets provides quantitative evidence for this claim.

The key **novel contribution** of this thesis is the matched total parameter budget methodology. While prior studies have compared models of different sizes or without controlling for model capacity, our approach isolates the effect of architecture from the effect of model size. The finding that ESNs achieve superior performance with only 1–10% trainable parameters (on synthetic data) provides strong evidence for the computational efficiency of the reservoir computing paradigm for well-characterized dynamical systems.

6 Conclusion

6.1 Summary of Key Findings

This thesis presented a systematic comparison of Echo-State Networks (implemented via the pyReCo library) and LSTM networks for chaotic time-series prediction under matched total parameter budgets. A total of 417 experiments were conducted across three benchmark datasets (Lorenz, Mackey-Glass, Santa Fe), three parameter budget levels ($\sim$1K, $\sim$10K, $\sim$50K), and varying data availability (1 000–10 000 samples), with five random seeds each and full statistical analysis.

The principal findings are:

1. **Performance is strongly dataset-dependent.** PyReCo significantly outperforms LSTM on synthetic chaotic systems—Lorenz (R^2 = 0.999 vs. 0.825) and Mackey-Glass (R^2 = 0.987 vs. 0.942)—while LSTM significantly outperforms PyReCo on the real-world Santa Fe laser dataset (R^2 = 0.934 vs. 0.770). All differences are statistically significant ($p < 0.05$) with large effect sizes (Cohen's $d > 2.0$).

2. **PyReCo is more data-efficient on synthetic data.** ESNs achieve high accuracy with as few as 1 000 training samples on the Lorenz and Mackey-Glass systems, while LSTMs require substantially more data to approach comparable performance. On the Santa Fe dataset, LSTM benefits more from additional data.

3. **Training efficiency exhibits a crossover.** When measured by training time with optimal parameters (the primary metric), PyReCo is up to 32$\times$ faster at small parameter budgets due to ridge regression training, but LSTM becomes 2.4$\times$ faster at large budgets due to the pyReCo library's quadratic scaling with reservoir size. LSTM additionally offers 50–560$\times$ faster inference across all budgets. However, PyReCo's hyperparameter tuning cost grows steeply with reservoir size (43 minutes at the large budget vs. 5 minutes for LSTM), reflecting the higher sensitivity of ESNs to hyperparameter choices.

4. **Multi-step prediction degrades universally.** Both models lose predictive power at long horizons (50 steps), consistent with the positive Lyapunov exponents of the benchmark systems. PyReCo maintains an advantage at short-to-medium horizons on synthetic data.

5. **Total parameter matching enables fair comparison.** By controlling for total model capacity rather than trainable parameters alone, this thesis demonstrates that ESNs achieve competitive or superior performance with only 1–10% trainable parameters on well-characterized dynamical systems.

6.2 Practical Recommendations

Based on the experimental evidence, the following recommendations guide model selection for chaotic time-series prediction:

Table 12: Practical model selection guide based on experimental findings.

Scenario	Recommended Model	Confidence
Known chaotic ODE/DDE system	PyReCo (ESN)	High
Synthetic time series, clean data	PyReCo (ESN)	High
Real-world sensor/experimental data	LSTM	High
Limited training data ($<$ 2 000 samples)	PyReCo (ESN)	Medium
Rapid retraining needed	PyReCo (ESN)	Medium
Latency-critical inference	LSTM	High
Large model budget ($>$ 50K params)	LSTM	Medium
Unknown data characteristics	Evaluate both	—

The choice between PyReCo and LSTM should be guided primarily by the *dataset characteristics*—whether the data comes from a known dynamical system or a complex real-world process—rather than by a blanket architectural preference.

6.3 Future Work

Several directions could extend the findings of this thesis:

- **Deep ESN architectures**: Multi-layer or hierarchical ESNs could potentially improve performance on complex real-world data by learning multi-scale representations, while retaining the fast training advantage.
- **Transformer comparison**: Attention-based architectures have shown strong performance on sequence tasks. Including Transformers in the comparison would provide a more complete picture of the architecture landscape.
- **Additional datasets**: Extending the benchmark to weather data, financial time series, and biological signals would test the generality of the dataset-dependent performance pattern.
- **Hybrid RC-LSTM architectures**: Combining a reservoir front-end with a trainable LSTM readout could potentially combine the strengths of both paradigms: fast feature extraction via reservoir dynamics and flexible pattern learning via gradient-based optimization.

- **Online/incremental learning**: Evaluating both architectures in a streaming setting, where the model must adapt to non-stationary dynamics, would be highly relevant for real-world deployment.
- **Trainable parameter matching**: Conducting a complementary study with matched trainable (rather than total) parameters would provide additional insight into the role of reservoir size versus optimization flexibility.

References

[1] Steven H Strogatz. *Nonlinear Dynamics and Chaos: With Applications to Physics, Biology, Chemistry, and Engineering*. CRC Press, 2 edition, 2018.

[2] Alan Wolf, Jack B Swift, Harry L Swinney, and John A Vastano. Determining Lyapunov exponents from a time series. *Physica D: Nonlinear Phenomena*, 16(3):285–317, 1985.

[3] Edward N Lorenz. Deterministic nonperiodic flow. *Journal of the Atmospheric Sciences*, 20(2):130–141, 1963.

[4] Michael C Mackey and Leon Glass. Oscillation and chaos in physiological control systems. *Science*, 197(4300):287–289, 1977.

[5] Andreas S Weigend and Neil A Gershenfeld. *Time Series Prediction: Forecasting the Future and Understanding the Past*. Addison-Wesley, Reading, MA, 1994.

[6] U Hübner, NB Abraham, and CO Weiss. Dimensions and entropies of chaotic intensity pulsations in a single-mode far-infrared NH_3 laser. *Physical Review A*, 40(11):6354–6365, 1989.

[7] Yoshua Bengio, Patrice Simard, and Paolo Frasconi. Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2):157–166, 1994.

[8] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. On the difficulty of training recurrent neural networks. *Proceedings of the 30th International Conference on Machine Learning*, pages 1310–1318, 2013.

[9] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

[10] Felix A Gers, Jürgen Schmidhuber, and Fred Cummins. Learning to forget: Continual prediction with LSTM. *Neural Computation*, 12(10):2451–2471, 2000.

[11] Klaus Greff, Rupesh K Srivastava, Jan Koutník, Bas R Steunebrink, and Jürgen Schmidhuber. LSTM: A search space odyssey. *IEEE Transactions on Neural Networks and Learning Systems*, 28(10):2222–2232, 2017.

[12] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.

[13] Michiel Hermans and Benjamin Schrauwen. Training and analysing deep recurrent neural networks. In *Advances in Neural Information Processing Systems*, volume 26, 2013.

[14] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *Proceedings of the 3rd International Conference on Learning Representations*, 2015.

[15] Dominic Masters and Carlo Luschi. Revisiting small batch training for deep neural networks. *arXiv preprint arXiv:1804.07612*, 2018.

[16] Wojciech Zaremba, Ilya Sutskever, and Oriol Vinyals. Recurrent neural network regularization. *arXiv preprint arXiv:1409.2329*, 2014.

[17] Lutz Prechelt. Early stopping — but when? In *Neural Networks: Tricks of the Trade*, pages 55–69. Springer, Berlin, Heidelberg, 1998.

[18] Mantas Lukoševičius and Herbert Jaeger. Reservoir computing approaches to recurrent neural network training. volume 3, pages 127–149. Elsevier, 2009.

[19] Gouhei Tanaka, Toshiyuki Yamane, Jean Benoit Héroux, Ryosho Nakane, Naoki Kanazawa, Seiji Takeda, Hidetoshi Numata, Daiju Nakano, and Akira Hirose. Recent advances in physical reservoir computing: A review. *Neural Networks*, 115:100–123, 2019.

[20] Herbert Jaeger. The “echo state” approach to analysing and training recurrent neural networks. Technical Report 148, German National Research Center for Information Technology, Bonn, Germany, 2001.

[21] Wolfgang Maass, Thomas Natschläger, and Henry Markram. Real-time computing without stable states: A new framework for neural computation based on perturbations. *Neural Computation*, 14(11):2531–2560, 2002.

[22] Mantas Lukoševičius. A practical guide to applying echo state networks. In *Neural Networks: Tricks of the Trade*, pages 659–686. Springer, Berlin, Heidelberg, 2012.

[23] Andrey N Tikhonov. Solution of incorrectly formulated problems and the regularization method. *Soviet Mathematics Doklady*, 4:1035–1038, 1963.

[24] Filip Matzner. Hyperparameter tuning in echo state networks. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO ’22)*, pages 404–412. ACM, 2022.

[25] Goodfellow E Mwamsojo, Allen R Mushi, and Neema N Mussa. A stochastic optimization technique for hyperparameter tuning in reservoir computing. *Neurocomputing*, 574:127262, 2024.

[26] Herbert Jaeger and Harald Haas. Harnessing nonlinearity: Predicting chaotic systems and saving energy in wireless communication. *Science*, 304(5667):78–80, 2004.

[27] Jaideep Pathak, Brian Hunt, Michelle Girvan, Zhixin Lu, and Edward Ott. Model-free prediction of large spatiotemporally chaotic systems from data: A reservoir computing approach. *Physical Review Letters*, 120(2):024102, 2018.

[28] Pantelis R Vlachas, Jaideep Pathak, Brian R Hunt, Themistoklis P Sapsis, Michelle Girvan, Edward Ott, and Petros Koumoutsakos. Backpropagation algorithms and reservoir computing in recurrent neural networks for the forecasting of complex spatiotemporal dynamics. *Neural Networks*, 126:191–217, 2020.

[29] Ashesh Chattopadhyay, Pedram Hassanzadeh, and Devika Subramanian. Data-driven predictions of a multiscale Lorenz 96 chaotic system using machine-learning methods: Reservoir computing, artificial neural network, and long short-term memory network. *Nonlinear Processes in Geophysics*, 27(3):373–389, 2020.

[30] Doreen Jirak, Stephan Tietz, Hazem Ali, and Stefan Wermter. Echo state networks and long short-term memory for continuous gesture recognition: A comparative study. *Cognitive Computation*, 15(5):1427–1439, 2023.

[31] Sunil Kumar and Rishi Kaushal. Forecasting crude oil prices using reservoir computing models. *Computational Economics*, 66(3):2543–2563, 2025.

[32] Oliver Wringe, Martin A Trefzer, and Susan Sherborne Stepney. Reservoir computing benchmarks: A review, a taxonomy, and a codebase. *International Journal of Parallel, Emergent and Distributed Systems*, 40(4):313–351, 2025.

[33] Yarin Gal and Zoubin Ghahramani. A theoretically grounded application of dropout in recurrent neural networks. *Advances in Neural Information Processing Systems*, 29, 2016.

[34] Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945.

[35] Jacob Cohen. Statistical power analysis for the behavioral sciences. 1988.

[36] Nils Bertschinger and Thomas Natschläger. Real-time computation at the edge of chaos in recurrent neural networks. *Neural Computation*, 16(7):1413–1436, 2004.

Appendix

A. PyReCo Hyperparameter Search Grids

The PyReCo hyperparameter grids are dataset-specific, reflecting the different optimal operating regions for each chaotic system. Each grid yields 36 combinations ($3 \times 3 \times 2 \times 2$), evaluated via 5-fold forward-chaining cross-validation.

Table 13: PyReCo hyperparameter grid for the Lorenz dataset.

Hyperparameter	Values
Spectral radius	0.8, 0.9, 1.0
Leakage rate	0.4, 0.5, 0.6
Density	0.05, 0.1
Input fraction	0.5, 0.75

Table 14: PyReCo hyperparameter grid for the Mackey-Glass dataset.

Hyperparameter	Values
Spectral radius	0.7, 0.8, 0.9
Leakage rate	0.2, 0.3, 0.4
Density	0.1, 0.15
Input fraction	0.5, 0.75

Table 15: PyReCo hyperparameter grid for the Santa Fe dataset.

Hyperparameter	Values
Spectral radius	0.7, 0.8, 0.9
Leakage rate	0.5, 0.6, 0.7
Density	0.05, 0.1
Input fraction	0.5, 0.75

B. Detailed Statistical Results

Table 16 presents the detailed statistical analysis for the Lorenz dataset at the default training fraction (0.7).

Table 16: Statistical comparison for Lorenz dataset (train fraction 0.7, $n = 5$ seeds).

Scale	Metric	PyReCo	LSTM	p-value	Cohen's d
Small	MSE	0.0042	0.0851	0.0001***	-10.3
	R^2	0.9856	0.7096	—	—
Medium	MSE	0.0027	0.0549	0.0015**	-4.7
	R^2	0.9908	0.8124	—	—
Large	MSE	0.0019	0.0609	0.0000***	-28.9
	R^2	0.9936	0.7922	—	—

*** $p < 0.001$, ** $p < 0.01$, * $p < 0.05$

Table 17: Statistical comparison for Mackey-Glass dataset (train fraction 0.7, $n = 5$ seeds).

Scale	Metric	PyReCo	LSTM	p-value	Cohen's d
Small	MSE	0.0180	0.1184	0.0567 ns	-1.7
	R^2	0.9792	0.8643	—	—
Medium	MSE	0.0179	0.0300	0.1315 ns	-1.2
	R^2	0.9794	0.9645	—	—
Large	MSE	0.0185	0.0656	0.0597 ns	-1.6
	R^2	0.9787	0.9265	—	—

*** $p < 0.001$, ** $p < 0.01$, * $p < 0.05$, ns = not significant

Table 18: Statistical comparison for Santa Fe dataset (train fraction 0.7, $n = 5$ seeds).

Scale	Metric	PyReCo	LSTM	p-value	Cohen's d
Small	MSE	0.2265	0.0797	0.0009***	5.7
	R^2	0.7963	0.9283	—	—
Medium	MSE	0.2507	0.0375	0.0001***	13.7
	R^2	0.7745	0.9663	—	—
Large	MSE	0.2521	0.2612	0.8066 ns	-0.2
	R^2	0.7733	0.7651	—	—

*** $p < 0.001$, ** $p < 0.01$, * $p < 0.05$, ns = not significant

C. Software and Hardware

All experiments were conducted using the following software stack:

- Python 3.11
- PyTorch (LSTM implementation)
- pyReCo library (ESN implementation, based on reservoirpy)
- scikit-learn (preprocessing, metrics)
- NumPy, SciPy (numerical computation)
- Matplotlib (visualization)

Random seeds (42, 43, 44, 45, 46) were fixed for all random number generators (Python, NumPy, PyTorch) to ensure full reproducibility. The complete source code and experiment scripts are available in the accompanying repository.